

Inside the LC-3 Virtual Machine

A line-by-line textbook on the x14 engine: how a 16-bit computer is modelled in modern C++, why every design choice was made, and how the tests prove it.

Repository	github.com/steverse/LC3
Source studied	src/lc3.hpp, src/lc3.cpp, src/main.cpp, tests/test_main.cpp, CMakeLists.txt
Documented on	2 October 2026
Build verified with	g++ 13.3 (C++17), Ubuntu 24.04; all 11 original test cases pass

NOTE — How to trust this book

Everything marked **VERIFIED HERE** was executed for this book: the code was compiled, the tests were run, and every worked example was fed through the real VM. Statements about *why* the author chose something are a reading of the code and its comments; where the code itself says nothing about intent, the text says so.

Contents

How to read this book	5
1 What is this project?	6
1.1 A computer inside a computer	6
1.2 Why the LC-3?	6
1.3 The project at a glance	6
1.4 The README's claims, mapped to code	7
2 The LC-3 machine	8
2.1 Memory: 65,536 words, not bytes	8
2.2 Registers	8
2.3 Condition codes: how the CPU remembers a comparison	9
2.4 Instruction encoding	9
2.5 The instruction set at a glance	10
2.6 Number-system primer: two's complement and hex	11
2.7 Addressing modes: four ways to name an operand	11
3 The build system, line by line	13
3.1 Why a static library?	14
3.2 The local CMakeLists versus the GitHub copy	14
3.3 The build pipeline	14
4 lc3.hpp: the blueprint	15
4.1 Includes: what each one is for	15
4.2 The storage: memory and registers	15
4.3 The vocabulary: five anonymous enums	16
4.4 The method declarations	17
5 Bit-level primitives	19
5.1 sign_extend	19
5.2 update_flags	20
5.3 swap and integer promotion	21
6 Loading programs: the .obj image	22
6.1 What is in an .obj file?	22
6.2 read_image and read_image_file	23
6.3 Loading several images and where execution begins	24
7 The fetch-decode-execute engine	25
7.1 cpu(): the whole life of a run	25
7.2 Fetch, decode, dispatch in three lines	26
7.3 step(): the same beat, one at a time	27
7.4 Watching a real program run	27
8 Dispatch by templates and bit masks	29

8.1 The table	29
8.2 What a function template is	29
8.3 The masks: one bit per opcode	30
8.4 Why dead code disappears	31
8.5 A subtle point: uninitialised locals	32
8.6 Alternatives and trade-offs	32
9 Every instruction, explained	34
9.1 Extracting the fields	34
9.2 Operate instructions: ADD, AND, NOT	34
9.3 Data movement: LD, LDI, LDR, LEA, ST, STI, STR	35
9.4 Control flow: BR, JMP/RET, JSR/JSRR	36
9.5 What is not implemented: RTI and opcode 13	37
9.6 The tail: updating the flags	37
10 System calls: the TRAP routines	39
10.1 The common prologue	40
10.2 Input traps and why read() replaces cin	40
10.3 Output traps	40
10.4 HALT and unknown traps	41
10.5 Things to know about the trap code	41
11 Memory-mapped keyboard input	42
11.1 The idea: devices that look like memory	42
11.2 memory_read: the device hook	42
11.3 check_key: asking without waiting	43
11.4 The polling idiom in LC-3 code	44
12 The terminal, signals, and shutdown	45
12.1 Why the terminal must be changed	45
12.2 Ctrl-C and the signal handler	46
12.3 Hardening opportunities in the signal path	46
13 Errors, lifecycle, and main()	47
13.1 main.cpp	47
13.2 Exceptions instead of exit()	47
14 Testing with Catch2	49
14.1 Catch2 in five minutes	49
14.2 The eleven test cases, decoded	49
14.3 Techniques worth copying	50
14.4 Results	51
14.5 What the suite does not cover	51
15 A critical review	53
15.1 Strengths	53

15.2 Findings	53
15.3 README versus code	54
16 Extending the machine: exercises	55
A Index of design decisions	56
B Cheat sheets	57
B.1 Hex, binary and powers of two	57
B.2 Field extraction recipes used in the code	57
B.3 Hand-assembling a program (the helper used for this book)	57
C Glossary	59
D Verification log	60
D.1 Build	60
D.2 Tests	60
D.3 Probes of suspected edge cases	60
D.4 Command-line behaviour	60
D.5 Interactive behaviour under a pseudo-terminal	61
D.6 Limits of this verification	61
E Added tests: tests/test_extra.cpp	62
F Complete source listings	65
F.1 CMakeLists.txt	65
F.2 src/lc3.hpp	65
F.3 src/lc3.cpp	67
F.4 src/main.cpp	72
F.5 tests/test_main.cpp	72

How to read this book

This is a textbook about one small program: a virtual machine that runs LC-3 machine code. The LC-3 ("Little Computer 3") is a teaching architecture from Patt and Patel's *Introduction to Computing Systems*. It is deliberately tiny, which makes it a perfect subject: the whole machine fits in your head, yet building it forces you to meet most of the ideas that make real computers work — binary encodings, two's complement, endianness, memory-mapped devices, terminal modes, signals, and the fetch-decode-execute loop.

The book is organised so that you can read it front to back, or jump to the instruction or mechanism you care about.

Part	Chapters	You will learn
I Foundations	1-2	What the project is; the LC-3 machine being imitated (memory, registers, instruction formats).
II Build & blueprint	3-4	The CMake build graph line by line; the class definition in <code>lc3.hpp</code> .
III Primitives	5-6	<code>sign_extend</code> , <code>update_flags</code> , <code>swap</code> ; loading <code>.obj</code> programs into memory.
IV The CPU	7-10	The execution loop, the template-based dispatch trick, every instruction, every TRAP routine.
V The outside world	11-13	Memory-mapped keyboard, raw terminal mode, signals, error handling, <code>main()</code> .
VI Verification	14-16	The Catch2 tests decoded by hand; a critical review with evidence; ideas for extending the VM.
Appendices	A-F	Cheat sheets, glossary, verification log, added tests, and the complete annotated source.

Conventions

CONCEPT

A **concept** box explains an idea from computer architecture or C++ that the code relies on.

DESIGN DECISION

A **design decision** box records a choice the author made, the alternatives, and the trade-off.

WATCH OUT

A **watch-out** box flags an edge case, limitation or bug. Each one was reproduced by running code, unless it says otherwise.

VERIFIED HERE

A **verified** box shows something that was actually executed while writing this book, with real output.

Numbers written like `0x3000` are hexadecimal. Register names are written `R0-R7`; the C++ names are `R_R0`, `R_PC` and so on. Source excerpts always show their true line numbers, so you can open the file next to the book.

A note on sources

The ground truth for this book is the code under study. The repository's README makes several claims about the engine; Chapter 1 maps each claim to the code that implements it, and Chapter 15 lists the few places where README wording and code differ. This is offered in the spirit of a code review: the project is a clean, well-tested first large C++ program, and the review is meant to help it get better.

PART I — FOUNDATIONS

1 What is this project?

1.1 A computer inside a computer

A **virtual machine** (here, an **emulator**) is a program that behaves like some other computer. Your laptop's processor understands x86-64 or ARM instructions; this program instead understands the 15 instructions of the LC-3. It keeps the imaginary computer's state — its memory, its registers, its program counter — in ordinary C++ variables, and it repeatedly does what the imaginary CPU would do: fetch an instruction, work out what it means, and change the state accordingly.

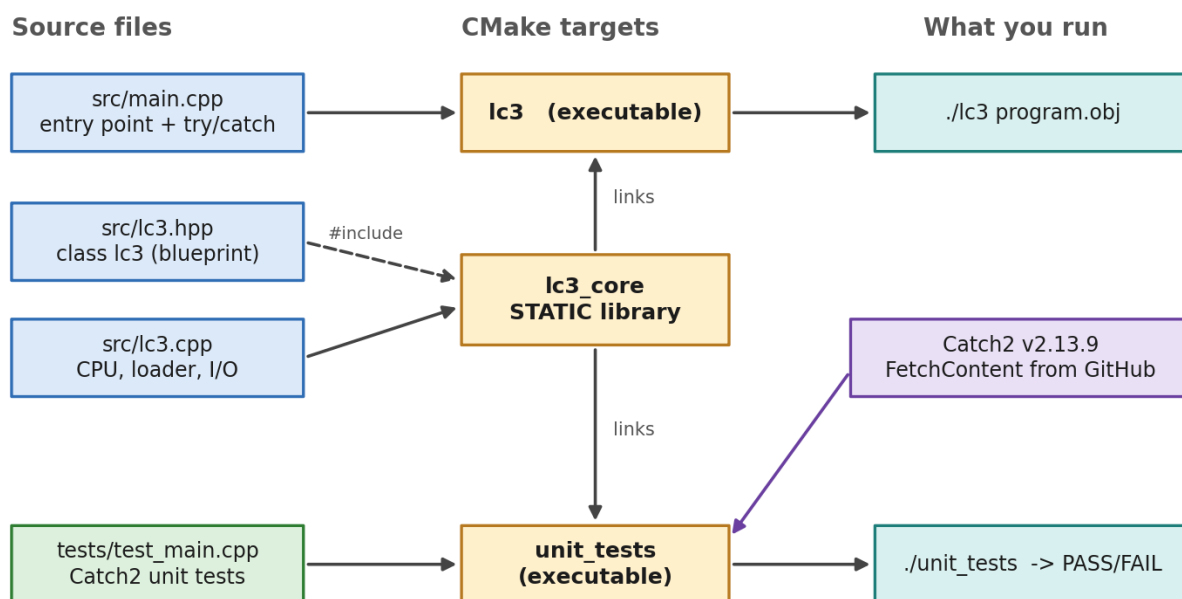
The README describes the result as a machine with "no operating system, no heap, no shortcuts": a 64K-word memory array, eight registers and a loop. That is accurate, and it is the single most useful sentence for understanding the code.

1.2 Why the LC-3?

Real instruction sets have hundreds of instructions and decades of accumulated quirks. The LC-3 keeps only what is essential: sixteen opcodes (fifteen used), eight general registers, three condition flags, and a 16-bit address space. It is small enough that a student can write a complete, correct emulator in a few hundred lines — which is exactly what this repository is. Programs written for the LC-3 (games such as 2048 or Rogue have been assembled for it by the community) run unmodified on such an emulator, so the project has a real, satisfying payoff.

1.3 The project at a glance

File	Lines	Role
src/lc3.hpp	129	The blueprint: class lc3 with the memory array, register file, and every named constant (opcodes, flags, trap codes).
src/lc3.cpp	423	The implementation: bit helpers, the templated instruction handlers, the run loop, image loader, keyboard I/O and terminal control.
src/main.cpp	19	Program entry: constructs the VM, runs it, converts any exception into a message and exit code 1.
tests/test_main.cpp	257	Catch2 test suite: 11 test cases using in-memory injection of machine code.
CMakeLists.txt	24	Build description: static library, test executable, Catch2 download.



Note: the uploaded CMakeLists.txt snapshot defines `lc3_core` and `unit_tests`; the GitHub copy also defines the `lc3` executable target.

Figure 1.1 — How the five files become three programs. The static library is built once and linked into both executables.

1.4 The README's claims, mapped to code

The README lists what the engine does. A good way to start reading unfamiliar code is to turn each claim into a question: *where in the source is this true?* The table gives the answers; later chapters explain each in depth.

README claim	Where it lives	Chapter
8 registers R0-R7, a PC, and condition flags N/Z/P; every operation mutates these	<code>enum + reg[R_COUNT]</code> in <code>lc3.hpp</code> ; every handler writes <code>vm->reg[...]</code>	2, 4, 9
Flat 64KB memory, no malloc/new, pre-allocated	<code>uint16_t memory[65536] = {0}</code> inside the object	4
Memory-mapped keyboard (KBSR ready bit, KBDR data)	<code>memory_read</code> special-cases the two addresses	11
Two's complement sign extension and endianness swapping	<code>sign_extend</code> , <code>swap</code>	5, 6
O(1) dispatch through a function-pointer array and templates	<code>op_table[16]</code> and <code>template <unsigned op> ins</code>	7, 8
Unbuffered, non-blocking I/O via POSIX <code>select()</code> and <code>termios</code>	<code>check_key</code> , <code>disable_input_buffering</code> , <code>restore_input_buffering</code>	11, 12
Exceptions instead of <code>std::exit()</code>	<code>lc3::cpu</code> throws; <code>main</code> catches	13
Catch2 tests with direct in-memory injection	<code>tests/test_main.cpp</code> uses <code>machine.step()</code> to run one instruction	14
Cycle-by-cycle stepping	<code>lc3::step()</code> executes exactly one instruction	7

CONCEPT

Instruction-accurate vs cycle-accurate. This VM executes one whole instruction per step. A *cycle-accurate* emulator would also model the internal micro-steps (fetch, decode, address evaluation, operand fetch, execute, store) as separate clock ticks. For running programs the difference is invisible; for studying the LC-3's internal state machine it is not. The README's phrase "cycle-by-cycle stepping" therefore means "instruction-by-instruction" here.

PART I — FOUNDATIONS

2 The LC-3 machine

Before reading the emulator, we need a clear picture of the machine it imitates. This chapter is the specification the code is trying to satisfy.

2.1 Memory: 65,536 words, not bytes

Most modern computers address memory in bytes. The LC-3 addresses memory in **16-bit words**: address 0x3000 names one 16-bit cell, and address 0x3001 names the next cell. There are $2^{16} = 65,536$ addresses, so the whole memory is 128 KiB. This is why the code declares `constexpr int MEMORY_SIZE = 1 << 16` and stores it in an array of `uint16_t`: the array index *is* the LC-3 address.

16-bit address space: 65,536 words (each word is 16 bits, NOT bytes)

0x0000 - 0x2FFF	plain RAM, zero-initialised. No OS image and no trap table are loaded (traps are host C++ code).
0x3000 PC_START	program entry point. .obj images are copied to their own origin. Code and data share this flat space.
... up to 0xFDFE	still ordinary RAM: stack, heap-like data, arrays...
0xFE00 KBSR	keyboard status register (bit 15 = 1 means a key is waiting)
0xFE02 KBDR	keyboard data register (reading it consumes the key)
0xFE03 - 0xFFFF	ordinary RAM again (no display registers, no MCR implemented)

(not to scale)

Figure 2.1 — The address space as this VM implements it. Only two addresses are special.

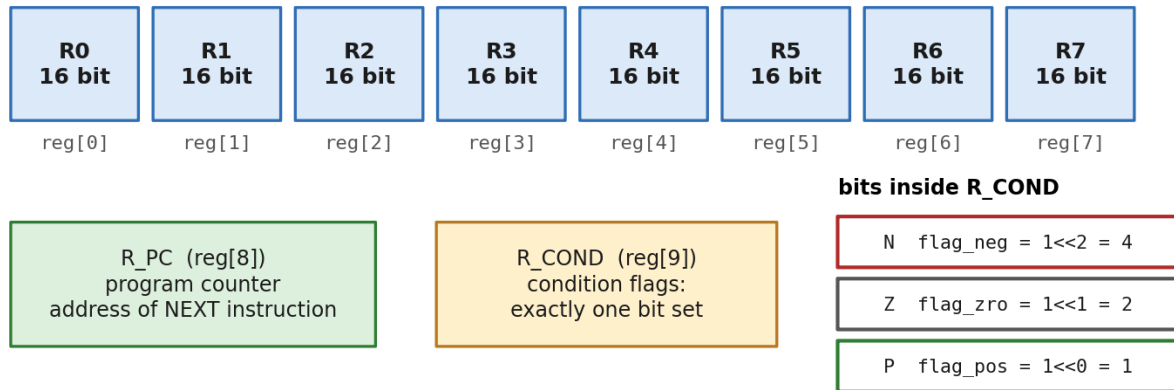
CONCEPT

Why a 16-bit index can never go out of range. An address is a `uint16_t`, so it can only hold values 0..65535 — exactly the legal indices of the array. The type system itself provides the bounds protection; no run-time check is needed for indexing `memory[address]`.

2.2 Registers

A **register** is a tiny, very fast storage cell inside the CPU. The LC-3 has eight general-purpose registers, R0 to R7, plus two special ones: the **program counter** (PC), which holds the address of the next instruction, and the **condition register**, which remembers whether the last result was negative, zero, or positive. The emulator keeps all ten in one array, `reg[10]`, and gives each a name through an enum.

General purpose (R7 doubles as the link register for JSR / TRAP)



R_COUNT = 10: the trailing enum entry is the array length, so reg[R_COUNT] cannot drift out of sync with the list.

Figure 2.2 — The ten registers held in reg[]. R7 has a conventional role as the return-address (link) register.

2.3 Condition codes: how the CPU remembers a comparison

The LC-3 has no "compare" instruction. Instead, every instruction that writes a value into a general register also updates three one-bit flags describing that value: **N** (negative: bit 15 is 1), **Z** (zero), **P** (positive). A later BR (branch) instruction tests those flags. To ask "is R3 > R4?" a program computes R3 - R4 into some register (an ADD/NOT/ADD sequence) and then branches on P. The three flags are mutually exclusive, so the code stores them as three distinct bit values in one register.

2.4 Instruction encoding

Every LC-3 instruction is exactly one 16-bit word. The top four bits (bits 15-12) are the **opcode** and choose one of 16 operations. The remaining twelve bits are operand fields whose meaning depends on the opcode. Figure 2.3 shows all formats. The code extracts fields with shifts and masks: for example (`instr >> 9) & 0x7` pulls out the three bits 11-9.

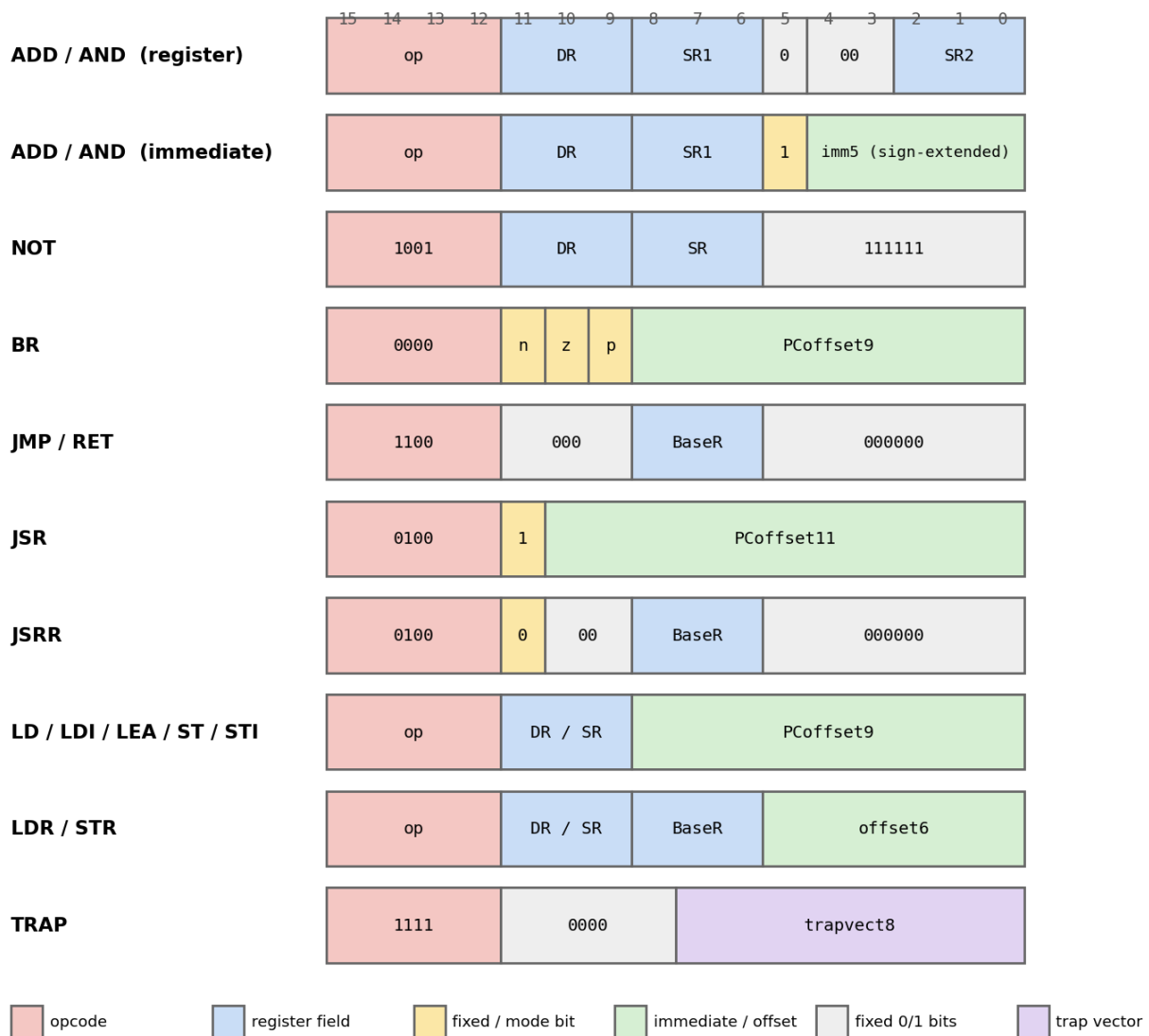


Figure 2.3 — Bit layout of every LC-3 instruction format (bit 15 on the left).

2.5 The instruction set at a glance

Op	Name	C++ enum	Operation	Sets N/Z/P?
0	BR	OP_BR	if (nzp & COND) PC = PC + sext(off9)	no
1	ADD	OP_ADD	DR = SR1 + (SR2 or sext(imm5))	yes
2	LD	OP_LD	DR = mem[PC + sext(off9)]	yes
3	ST	OP_ST	mem[PC + sext(off9)] = SR	no
4	JSR / JSRR	OP_JSR	R7 = PC; PC = PC + sext(off11) or PC = BaseR	no
5	AND	OP_AND	DR = SR1 & (SR2 or sext(imm5))	yes
6	LDR	OP_LDR	DR = mem[BaseR + sext(off6)]	yes
7	STR	OP_STR	mem[BaseR + sext(off6)] = SR	no
8	RTI	OP_RTI	return from interrupt (not implemented here)	n/a
9	NOT	OP_NOT	DR = ~SR	yes
10	LDI	OP_LDI	DR = mem[mem[PC + sext(off9)]]	yes
11	STI	OP_STI	mem[mem[PC + sext(off9)]] = SR	no
12	JMP / RET	OP_JMP	PC = BaseR (RET is JMP R7)	no

Op	Name	C++ enum	Operation	Sets N/Z/P?
13	(reserved)	OP_RES	unused opcode	n/a
14	LEA	OP_LEA	DR = PC + sext(off9) (address, not contents)	yes *
15	TRAP	OP_TRAP	R7 = PC; run system routine trapvect8	no **

* Editions of the LC-3 specification differ on whether LEA updates the condition codes. This VM does update them (mask 0x4666, bit 14). ** GETC and IN set the flags explicitly inside their trap handlers.

2.6 Number-system primer: two's complement and hex

Computers store negative numbers in **two's complement**: in an n -bit field, the top bit has weight -2^{n-1} instead of $+2^{n-1}$. So in 5 bits, 11101 means $-16 + 8 + 4 + 0 + 1 = -3$. A useful consequence: adding, subtracting and multiplying work identically for signed and unsigned values, and overflow simply wraps around modulo 2^n . That is why the emulator can use ordinary `uint16_t` addition for ADD and get the hardware's behaviour for free.

5-bit pattern	As unsigned	As two's complement	16-bit sign-extended
00000	0	0	0x0000
01111	15	+15 (largest)	0x000F
10000	16	-16 (smallest)	0xFFFF0
11101	29	-3	0xFFFFD
11111	31	-1	0xFFFFF

Hexadecimal is shorthand for binary: each hex digit is exactly four bits. The instruction 0x1042 is 0001 0000 0100 0010, and the leading hex digit (1) *is* the opcode. Appendix B has the full nibble table.

2.7 Addressing modes: four ways to name an operand

An instruction must say *where* its data is. The LC-3 offers register, immediate, PC-relative, indirect, and base+offset forms. Because offsets are small (5, 6, 9 or 11 bits), the code sign-extends them to 16 bits before adding.

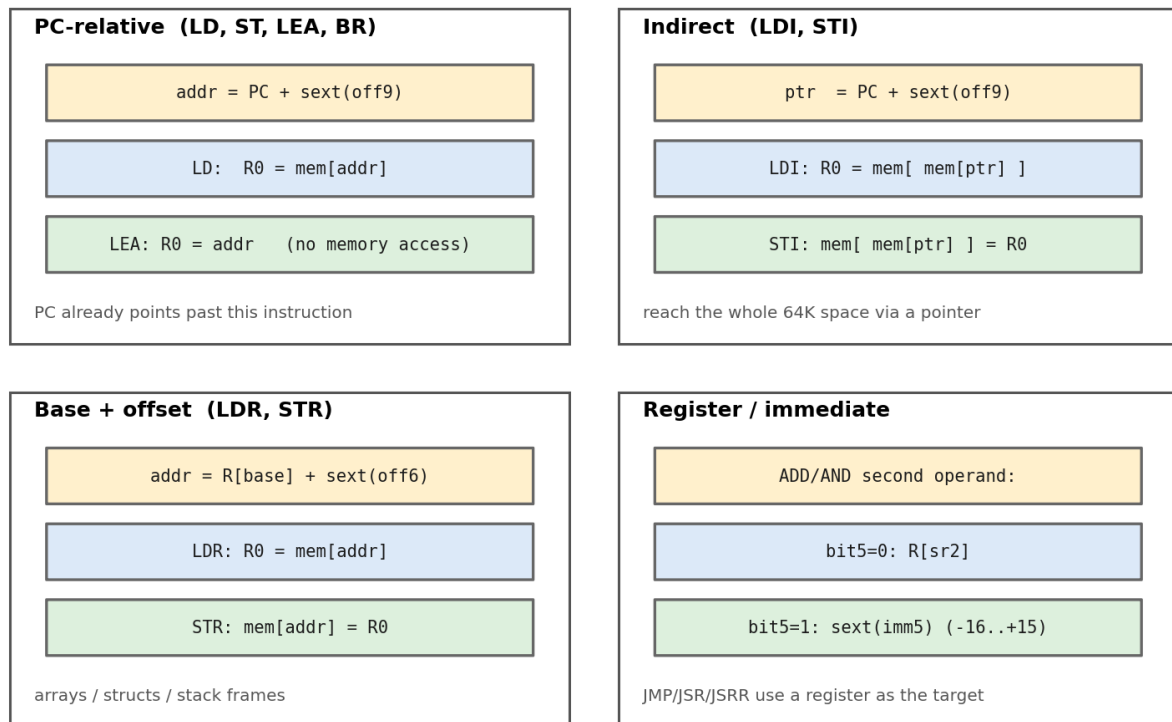


Figure 2.4 — The addressing modes and which opcodes use each.

PART II — BUILD SYSTEM AND BLUEPRINT

3 The build system, line by line

CMake does not compile code itself. It reads a description of **targets** (things to build) and their relationships, and generates the build files for your platform (Makefiles, Ninja, Visual Studio...). The whole description of this project is 24 lines.

CMakeLists.txt (lines 1-24)

```
1 cmake_minimum_required(VERSION 3.15...4.1)
2 project(x14_lc3 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 #Define the virtual target
6 add_library(lc3_core STATIC src/lc3.cpp) #interphase vs dynamic vs static
7
8 #Map the virtual target to the 'src' directory (linking/shows the preprocessor where to look for
  » when it sees an unknown #include directory eg #include<lc3.hpp>)
9 target_include_directories(lc3_core PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/src)
10
11
12
13 # Download Catch2 directly from GitHub
14 include(FetchContent)
15 FetchContent_Declare(
16   Catch2
17   GIT_REPOSITORY https://github.com/catchorg/Catch2.git
18   GIT_TAG        v2.13.9
19 )
20 FetchContent_MakeAvailable(Catch2)
21
22 # Assemble the test binary target and link it
23 add_executable(unit_tests tests/test_main.cpp)
24 target_link_libraries(unit_tests PRIVATE lc3_core Catch2)
```

Line(s)	What it says	Why it matters
cmake_minimum_required(VERSION 3.15...4.1)	Needs CMake 3.15 or newer; the <i>a...b</i> range tells CMake to apply behaviour of policies up to 4.1 when available.	Newer CMake versions warn about very old minimum versions; the range silences that while still supporting 3.15.
project(x14_lc3 LANGUAGES CXX)	Names the project (x14 is the engine's internal codename) and enables only the C++ compiler.	Skipping C avoids a needless compiler check.
set(CMAKE_CXX_STANDARD 17)	Compile with <code>-std=c++17</code> (or the closest supported).	The code needs nothing newer than C++11/17 features. README says C++20; the build asks for 17.
add_library(lc3_core STATIC src/lc3.cpp)	Compile lc3.cpp into a static library (a .a archive). The comment in the file lists the alternatives: STATIC, SHARED, INTERFACE.	Both the game executable and the test executable need the same VM code; building it once and linking twice avoids compiling it twice.
target_include_directories(lc3_core PUBLIC \${CMAKE_CURRENT_SOURCE_DIR}/src)	Add src/ to the header search path. PUBLIC means anything that links lc3_core inherits it.	This is why tests can write <code>#include "lc3.hpp"</code> though the test lives in tests/.
FetchContent_Declare(Catch2 ... GIT_TAG v2.13.9)	Download the Catch2 test framework at configure time, pinned to an exact release tag.	Pinning makes builds reproducible. v2 is the single-header generation (catch2/catch.hpp).
add_executable(unit_tests tests/test_main.cpp)	Build a test program from one source file.	Contains its own main() (generated by CATCH_CONFIG_MAIN).
target_link_libraries(unit_tests PRIVATE lc3_core Catch2)	Link the tests against the VM library and Catch2.	PRIVATE: nothing else links the test binary, so nothing needs to inherit these.

3.1 Why a static library?

Look at the alternatives the author's own comment hints at ("interphase vs dynamic vs static", meaning INTERFACE vs SHARED vs STATIC):

Kind	What it produces	Fit for this project?
STATIC	liblc3_core.a, an archive of object code copied into each executable at link time.	Yes. No runtime dependency, simple, and the test binary and the game binary each carry the same code.
SHARED	liblc3_core.so, loaded at run time.	Unnecessary complexity for a small program with two consumers.
INTERFACE	Nothing compiled — only usage requirements (include paths, flags). Right for header-only libraries.	No: lc3.cpp contains real code.

DESIGN DECISION

Splitting the program into a library plus thin executables. main.cpp is 18 lines and contains the only `main()`. Everything testable lives in the library, so the unit tests can construct an `lc3` object and poke it directly without going through the command line. This separation is the single most important structural decision in the repository: it is what makes the VM testable.

3.2 The local CMakeLists versus the GitHub copy

WATCH OUT

The CMakeLists.txt studied here (24 lines) defines `lc3_core` and `unit_tests` but **no executable for main.cpp**. The copy currently shown on GitHub (31 lines) additionally contains `add_executable(lc3 src/main.cpp)` followed by `target_link_libraries(lc3 PRIVATE lc3_core)`, which is what the README's step 3 (`cmake --build . --target lc3`) needs. If your local build has no `lc3` target, add those two lines.

```
# add to the end of CMakeLists.txt
add_executable(lc3 src/main.cpp)
target_link_libraries(lc3 PRIVATE lc3_core)
```

3.3 The build pipeline

Step	Command (from the README)	What happens
Configure	<code>cmake -DCMAKE_BUILD_TYPE=Release ..</code>	CMake reads CMakeLists.txt, downloads Catch2 with git, writes Makefiles. Release adds <code>-O3 -DNDEBUG</code> .
Build tests	<code>cmake --build . --target unit_tests</code>	Compiles <code>lc3.cpp</code> into <code>liblc3_core.a</code> , compiles <code>test_main.cpp</code> , links them with Catch2.
Run tests	<code>./unit_tests</code>	Catch2 runs every <code>TEST_CASE</code> and prints a summary.
Build VM	<code>cmake --build . --target lc3</code>	Compiles <code>main.cpp</code> and links it to the same library.
Run a program	<code>./lc3 path/to/payload.obj</code>	Loads the image and executes it.

VERIFIED HERE

What was run for this book. The build environment had no CMake and no network, so *CMake itself was not executed*. The graph was rebuilt by hand with the same settings (C++17, include path `src/`): `g++ -std=c++17 -c src/lc3.cpp; ar rcs liblc3_core.a lc3.o; g++ tests/test_main.cpp liblc3_core.a -Isrc; g++ src/main.cpp liblc3_core.a -Isrc`. The tests were linked against a small stand-in for `catch2/catch.hpp` because the real Catch2 could not be downloaded. Result: **all 11 test cases (36 assertions) passed**, also under AddressSanitizer + UBSan. Please run the real CMake flow once on your machine to confirm.

PART II — BUILD SYSTEM AND BLUEPRINT

4 lc3.hpp: the blueprint

The header declares *what a machine is*; the .cpp file says *what it does*. Read the header first: once you know the state and the vocabulary, the logic is easy to follow.

4.1 Includes: what each one is for

src/lc3.hpp (lines 5-18)

```
5 #pragma once
6
7 #include <cstdio>
8 #include <cstdint>
9 #include <csignal>
10 #include <cstdlib>
11 #include <unistd.h>
12 #include <fcntl.h>
13 #include <termios.h>
14 #include <iostream>
15 #include <sys/time.h>
16 #include <sys/types.h>
17 #include <sys/mman.h>
18 #include <sys/select.h>
```

Header	Used for	Where
<cstdio>	FILE, fopen, fread, fclose, printf	image loading, interrupt handler
<cstdint>	uint16_t (exactly 16 bits)	everywhere
<csignal>	std::signal, SIGINT	cpu()
<cstdlib>	general utilities (nothing critical used)	—
<unistd.h>	read(), STDIN_FILENO (POSIX)	keyboard reads
<termios.h>	struct termios, tcgetattr/tcsetattr	raw terminal mode
<sys/select.h>, <sys/time.h>	select(), fd_set, struct timeval	non-blocking key poll
<iostream>	std::cout (also pulls in std::string and exception types)	trap output
<fcntl.h>, <sys/mman.h>, <sys/types.h>	not needed by the current code	harmless leftovers

CONCEPT

#pragma once tells the compiler to include this file at most once per translation unit, replacing the older three-line include-guard idiom. It is not in the C++ standard but every mainstream compiler supports it.

NOTE

Because , and are POSIX headers, the engine builds on Linux and macOS (and WSL) but not natively on Windows. The README says as much by calling the I/O layer "POSIX".

4.2 The storage: memory and registers

src/lc3.hpp (lines 20-47)

```
20 constexpr int MEMORY_SIZE = 1 << 16;
21
22 class lc3
23 {
24     // have to impose boundary protection and initialize the registers
25 public:
26     // STORAGE
27
28     uint16_t memory[MEMORY_SIZE] = {0}; // memory array
```

```

29
30 // registers
31 enum
32 {
33
34     R_R0 = 0,
35     R_R1,
36     R_R2,
37     R_R3,
38     R_R4,
39     R_R5,
40     R_R6,
41     R_R7,
42     R_PC, // program counter
43     R_COND, // condition flags
44     R_COUNT // automatically counts the number of registers
45 };
46
47 uint16_t reg[R_COUNT] = {0}; // register array

```

Three details deserve attention.

- `MEMORY_SIZE` is a compile-time constant. `1 << 16` equals 65,536 exactly, so array size and address range always agree.
- `uint16_t memory[MEMORY_SIZE] = {0}` puts 128 KiB *inside every lc3 object*. There is no `new` and no `malloc`; the memory exists exactly as long as the object does. The initialiser `= {0}` zero-fills it (an uninitialised array would contain garbage from earlier stack use, making runs non-deterministic).
- The register enum ends with `R_COUNT`. Because enum values count up from 0, `R_COUNT` automatically equals the number of registers (10). Adding a register above it updates the array size with no other edit. The test suite even asserts `lc3::R_COUNT == 10`.

DESIGN DECISION

Everything is public. The class has no private section; tests read and write `memory` and `reg` directly. That is a conscious trade: it gives up encapsulation in return for very direct white-box tests ("put this word in memory, step once, check that register"). A larger project would usually expose a narrow interface instead; for an emulator whose *whole point* is to expose machine state, public members are defensible.

VERIFIED HERE

Size of the object. `sizeof(lc3)` printed **131,156 bytes** = 131,072 (memory) + 20 (ten `uint16_t` registers) + 60 (struct `termios` on Linux/glibc) + 4 (int running). This is a large object for the stack; it works because typical stacks are 8 MiB. Creating many VMs on threads with small stacks would need heap allocation.

4.3 The vocabulary: five anonymous enums

src/lc3.hpp (lines 84-128)

```

84     OP_BR = 0, // branch
85     OP_ADD,   // add
86     OP_LD,    // load
87     OP_ST,    // store
88     OP_JSR,   // jump register
89     OP_AND,   // bitwise and
90     OP_LDR,   // load register
91     OP_STR,   // store register
92     OP_RTI,   // unused
93     OP_NOT,   // bitwise not
94     OP_LDI,   // load indirect
95     OP_STI,   // store indirect
96     OP_JMP,   // jump
97     OP_RES,   // reserved (unused)
98     OP_LEA,   // load effective address
99     OP_TRAP  // execute trap
100 };
101
102 // conditional flags
103 // enum is a compile-time literal so no overhead
104 enum
105 {
106
107     flag_pos = 1 << 0,

```



```

108         flag_zro = 1 << 1,
109         flag_neg = 1 << 2
110     };
111
112     enum
113     {
114         MR_KBSR = 0xFE00,
115         MR_KBDR = 0xFE02
116     };
117
118     enum
119     {
120         TRAP_GETC = 0x20,
121         TRAP_OUT = 0x21,
122         TRAP_PUTS = 0x22,
123         TRAP_IN = 0x23,
124         TRAP_PUTSP = 0x24,
125         TRAP_HALT = 0x25
126     };
127
128     int running = 1;

```

The header defines five unnamed enums: registers, opcodes, condition flags, memory-mapped register addresses, and trap vectors. The comment beside the flags enum gives the reason: "enum is a compile-time literal so no overhead". An enumerator is just an integer constant that the compiler substitutes; it takes no storage and no run-time cost, unlike a `static const int` variable, which might occupy memory and require a load.

Enum	Values	Encoded how
registers	R_R0..R_R7 = 0..7, R_PC = 8, R_COND = 9, R_COUNT = 10	index into reg[]
opcodes	OP_BR = 0 ... OP_TRAP = 15	equals the 4-bit opcode, so op = instr >> 12 indexes it directly
flags	flag_pos = 1<<0, flag_zro = 1<<1, flag_neg = 1<<2	one-hot bits; the same bit order as the n/z/p bits inside a BR instruction
memory-mapped	MR_KBSR = 0xFE00, MR_KBDR = 0xFE02	addresses the CPU treats as devices
traps	TRAP_GETC 0x20, OUT 0x21, PUTS 0x22, IN 0x23, PUTSP 0x24, HALT 0x25	the low 8 bits of a TRAP instruction

CONCEPT

Why the flag order matters. Inside a BR instruction the three condition bits are n (bit 11), z (bit 10), p (bit 9). Shifting the instruction right by 9 and masking 7 gives the value `(instr >> 9) & 7` with p in bit 0, z in bit 1, n in bit 2. The flag enum uses *the same order*, so the branch test is a single AND: `cond & reg[R_COND]`. No translation table is needed.

4.4 The method declarations

src/lc3.hpp (lines 49-78)

```

49     // CPU
50
51     // sign extend
52     uint16_t sign_extend(uint16_t x, int bit_count);
53
54     // flag update
55     void update_flags(uint16_t r);
56
57     // cpu (execution loop)
58     void cpu(int argc, const char *argv[]);
59
60     // swap to account for Endianness
61     uint16_t swap(uint16_t x);
62
63     // read the image file (.obj)
64     int read_image(char const *image_path);
65
66     // reading image file into memory
67     void read_image_file(FILE *file);

```

```

68
69     void memory_write(uint16_t adress, uint16_t val);
70
71     static void handle_interrupt(int signal);
72
73     struct termios original_tio;
74     uint16_t memory_read(uint16_t adress);
75     void disable_input_buffering();
76     void restore_input_buffering();
77     uint16_t check_key();
78     void step(); // webassm

```

A few remarks on the declaration list:

- `step()` has a trailing comment `// webassm`. The single-step method exists so that a host can drive the machine one instruction at a time — for instance a browser (WebAssembly) frame loop, a debugger, or the unit tests.
- `handle_interrupt` is `static`. A signal handler is called by the operating system with no object; it must be a plain function or a static member (which has no hidden `this` pointer). That is why the file also has a global `running_vm` — the handler needs some way to find the machine (Chapter 12).
- `struct termios original_tio` is a data member of the class because the terminal settings belong to *this* machine's session and must be restored exactly as found.
- The parameter is spelled `adress` in the header but `address` in the .cpp file — C++ only cares about the types, so both are legal; it is a typo, not a bug.

PART III — THE PRIMITIVES

5 Bit-level primitives

Three tiny functions carry most of the machine's low-level correctness: `sign_extend` widens small signed fields, `update_flags` maintains the N/Z/P flags, and `swap` repairs byte order. Each is short enough to verify by hand, and each has a dedicated unit test.

5.1 `sign_extend`

Instruction fields such as `imm5` or `PCoffset9` are signed numbers packed into few bits. To add them to a 16-bit register they must first be widened to 16 bits *without changing their value*. For a positive number, fill the new upper bits with 0. For a negative number, fill them with 1. The rule is: **copy the sign bit upward**.

src/lc3.cpp (lines 6-20)

```
6  uint16_t lc3::sign_extend(uint16_t x, int bit_size)
7  {
8
9      if (x >> (bit_size - 1) & 1)
10     {
11
12         x = (x | (0xFFFFU << bit_size));
13     }
14
15     else
16     {
17         x = 0xFFFFU & x;
18     }
19     return x;
20 }
```

Reading the code:

- `x >> (bit_size - 1) & 1` isolates the sign bit (the top bit of the `bit_size`-wide field). Operator precedence in C++ ranks shift above bitwise AND, so this is `(x >> (bit_size - 1)) & 1`.
- If it is 1, `x | (0xFFFFU << bit_size)` builds a mask of ones from bit `bit_size` upward, and OR sets all those bits. The constant is written `0xFFFFU` (unsigned) so that shifting cannot overflow a signed int, which would be undefined behaviour.
- If it is 0, the else-branch `x = 0xFFFFU & x` just keeps the low 16 bits. It is effectively a no-op (callers already pass masked values) but documents the intent and keeps the two branches symmetrical.

`sign_extend(0x1D, 5)` - `imm5 = 11101b` = -3 in 5-bit two's complement

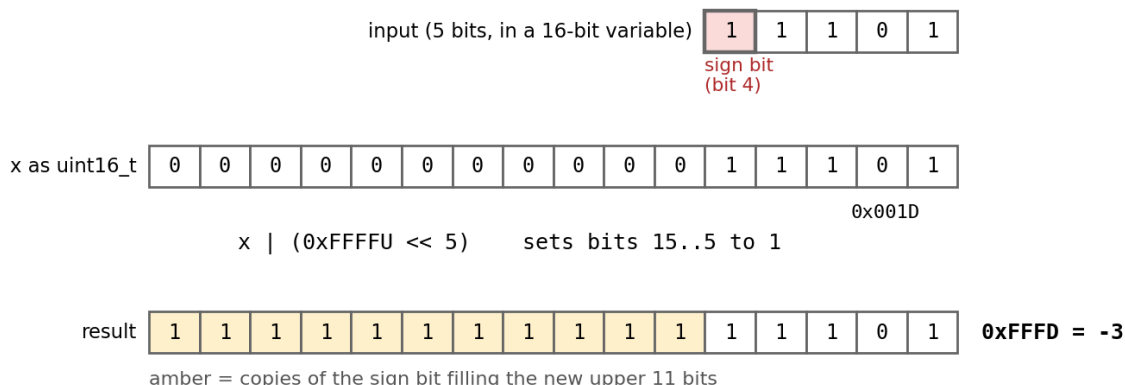


Figure 5.1 — `sign_extend(0x1D, 5)`: the 5-bit pattern 11101 (-3) becomes 0xFFFF (-3 in 16 bits).

Call	Field bits	Sign bit	Result	Meaning
<code>sign_extend(0x0A, 4)</code>	1010	1	0xFFFFA	-6 stays -6

Call	Field bits	Sign bit	Result	Meaning
sign_extend(0x00, 5)	00000	0	0x0000	0
sign_extend(0x0F, 5)	01111	0	0x000F	+15 (largest imm5)
sign_extend(0x10, 5)	10000	1	0xFFFF0	-16 (smallest imm5)
sign_extend(0x1F, 5)	11111	1	0xFFFF	-1

VERIFIED HERE

These five rows are exactly the assertions in the *ALU Bitwise Math Validation* test, and they pass.

CONCEPT

An equivalent trick. Many emulators write this as `((x & (1<<(n-1))) ? x | (0xFFFF << n) : x)`. It is shorter but harder to read. The author's branching version states the rule directly, which is the better choice for a learning project.

5.2 update_flags

src/lc3.cpp (lines 22-41)

```

22 void lc3::update_flags(uint16_t r)
23 {
24     if (reg[r] == 0)
25     {
26         reg[R_COND] = flag_zro;
27     }
28
29     else if (reg[r] >> 15 & 1)
30     {
31         reg[R_COND] = flag_neg;
32     }
33
34     else
35     {
36         reg[R_COND] = flag_pos;
37     }
38
39 }
40
41 }
```

The parameter `r` is a *register index*, not a value: the function reads `reg[r]` itself. It tests in a careful order — zero first, then the sign bit (bit 15), otherwise positive — and **assigns** (not ORs) the result, guaranteeing that exactly one flag is set at any time.

Value in register	Test that fires	R_COND becomes
0x0000	<code>== 0</code>	flag_zro (2)
0x0001 ... 0x7FFF	sign bit 0	flag_pos (1)
0x8000 ... 0xFFFF	sign bit 1	flag_neg (4)

CONCEPT

Zero must be tested first. Zero has sign bit 0, so checking the sign bit first would misclassify it as positive. The order of the if-chain is therefore part of the specification.

5.3 swap and integer promotion

src/lc3.cpp (lines 297-301)

```
297 uint16_t lc3::swap(uint16_t x)
298 {
299
300     return (x << 8) | (x >> 8);
301 }
```

swap exchanges the two bytes of a 16-bit word. In C++, arithmetic on `uint16_t` values is performed after **integer promotion** to `int`, so `x << 8` can momentarily exceed 16 bits (`0x1234 << 8 = 0x123400`). Converting the final result back to `uint16_t` on return truncates it to the low 16 bits, giving `0x3400`, which OR-ed with `x >> 8` (`0x0012`) yields `0x3412`. The same promote-then-truncate behaviour is what makes ADD wrap around like real hardware.

Input	x << 8 (as int)	x >> 8	OR, truncated to 16 bits
0x1234	0x123400	0x0012	0x3412
0x00FF	0xFF00	0x0000	0xFF00
0xABCD	0xABCD00	0x00AB	0xCDAB

VERIFIED HERE

Integer wrap-around, run for real. `ADD R1,R1,#1` executed with `R1 = 0x7FFF` gave `R1 = 0x8000` and `COND = N (4)`: the largest positive 16-bit signed value plus one becomes the most negative, exactly as a hardware adder behaves. No overflow flag exists on the LC-3, and none is emulated.

6 Loading programs: the .obj image

6.1 What is in an .obj file?

A compiled LC-3 program is a tiny binary file: one **origin** word saying where in memory to place the program, followed by the program's words. Both are stored **big-endian** (most significant byte first), the convention of the original LC-3 tools.

VERIFIED HERE

A real image. The first bytes of the hello.obj built for this book (a program that prints a string then halts) are:

```
30 00 E0 02 F0 22 F0 25 00 48 00 65 00 6C 00 6C ...
```

Bytes 30 00 are the origin 0x3000; E0 02 is the first instruction (LEA R0, +2); F0 22 is TRAP x22 (PUTS); F0 25 is TRAP x25 (HALT); 00 48 is 'H' stored as a 16-bit word.

hello.obj on disk (big-endian words)

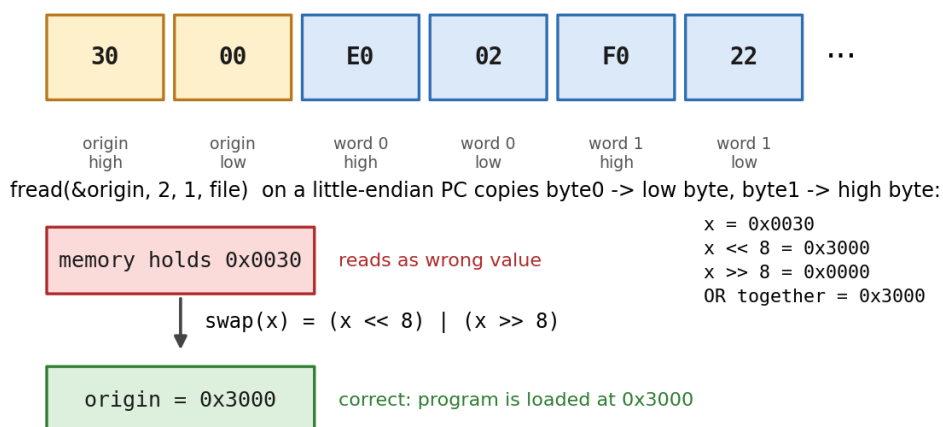


Figure 6.1 — Big-endian words on disk, read by `fread()` on a little-endian host, repaired by `swap()`.

Almost every desktop CPU today (x86-64, and ARM in its usual mode) is **little-endian**: the low byte of a multi-byte value comes first in memory. So `fread(&origin, 2, 1, file)` copies the file's first byte into the *low* half of the variable and the second into the high half — the opposite of what the file means. `swap` corrects that after each read.

WATCH OUT

The unconditional swap assumes a little-endian host. On a big-endian machine the file would already be in the right order and the swap would corrupt it. A portable loader would swap only when the host is little-endian (or assemble each word from two bytes explicitly).

6.2 read_image and read_image_file

src/lc3.cpp (lines 303-312)

```
303 int lc3::read_image(char const *image_path)
304 {
305     FILE *file = fopen(image_path, "rb");
306     if (!file)
307         return 0;
308     else
309         read_image_file(file);
310     fclose(file);
311     return 1;
312 }
```

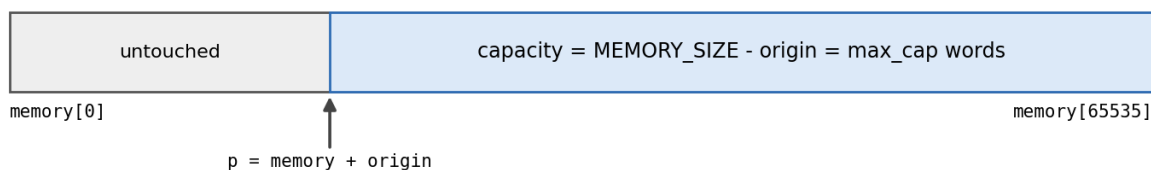
src/lc3.cpp (lines 314-333)

```
314 void lc3::read_image_file(FILE *file)
315 {
316     uint16_t orgin;
317     if (fread(&orgin, sizeof(orgin), 1, file) != 1) // reads count of elements found
318     {
319         throw std::runtime_error("Corrupted or empty image file ");
320     }
321     orgin = swap(orgin);
322
323     uint16_t max_cap = MEMORY_SIZE - orgin;
324     uint16_t *p = memory + orgin; // advances the pointer by origin elements – not origin bytes
    » (also note pointer decay)
325
326     size_t read = fread(p, sizeof(uint16_t), max_cap, file);
327
328     while (read-- > 0)
329     {
330         *p = swap(*p);
331         ++p;
332     }
333 }
```

Step by step

- `fopen(path, "rb")` opens in binary mode ("rb"). Text mode on some platforms would translate newline bytes and corrupt the image. It returns 0 if the file cannot be opened; the caller turns that into an exception (Chapter 13).
- `fread(&orgin, sizeof(orgin), 1, file)` returns the number of *items* read (here 0 or 1). Anything but 1 means an empty or truncated file, so the function throws `std::runtime_error`.
- After swapping, `max_cap = MEMORY_SIZE - orgin` computes how many words fit between the origin and the end of memory.
- `uint16_t *p = memory + orgin` is **pointer arithmetic**: adding an integer to a `uint16_t*` advances by that many *elements* (2 bytes each), not bytes. The array name `memory` *decays* to a pointer to element 0 in this expression; that is the "pointer decay" the author notes in the comment.
- One `fread` call reads up to `max_cap` words straight into memory. It stops early at end of file, and its return value is the number of words actually read — which sizes the swap loop.
- The loop `while (read-- > 0) { *p = swap(*p); ++p; }` swaps every loaded word in place. Only words actually present are swapped; the rest of memory stays zero.

read_image_file: where the words land



- 1) `fread(&origin) -> swap -> origin`
- 2) `p = memory + origin` (pointer arithmetic counts `uint16_t` elements, not bytes)
- 3) `read = fread(p, 2, max_cap, file)` (stops at EOF: 'read' = words actually present)
- 4) for each of the 'read' words: `*p = swap(*p); ++p`

Figure 6.2 — Destination and capacity of the load.

6.3 Loading several images and where execution begins

Inside `lc3::cpu`, the loop `for (int j = 1; j < argc; j++)` loads every command-line argument in order. Images are simply copied to their own origins, so a program and a separate data image can coexist; an image that overlaps an earlier one overwrites it. Execution always begins at `0x3000` regardless of any origin (the conventional LC-3 user-program start).

WATCH OUT

An image with origin `0x0000` loads nothing. `MEMORY_SIZE - origin` is an `int` (65,536) assigned into a `uint16_t`. For every origin other than 0 the value fits; for origin 0 it truncates to 0, and `fread` then reads zero words. *Reproduced:* a file with origin 0 and one word `0x1234` left `memory[0] = 0x0000`. Typical programs start at `0x3000` so this never shows up in practice. Fix: declare `size_t max_cap`.

WATCH OUT

If `read_image_file` throws (empty file), the `FILE*` opened by `read_image` is not closed, because `fclose` comes after the call. The process exits soon after so nothing visibly breaks, but a small RAIL wrapper (or `std::unique_ptr<FILE, decltype(&fclose)>`) would make it leak-proof.

VERIFIED HERE

Loader test. The *Image Reading and File Loading* test writes a real 4-byte file (origin `0x3000`, word `0x1234`, both swapped to big-endian), calls `read_image`, and checks `memory[0x3000] == 0x1234`. An additional test written for this book loads two words at origin `0x4000` from raw bytes `40 00 AB CD 12 34` and confirms they land at `0x4000` and `0x4001`.

PART IV — THE CPU

7 The fetch-decode-execute engine

Every CPU ever built runs the same three-beat loop: **fetch** the instruction the program counter points at, **decode** what it means, **execute** it. In this VM that loop is short enough to read in one sitting.

7.1 cpu(): the whole life of a run

src/lc3.cpp (lines 251-295)

```
251 void lc3::cpu(int argc, const char *argv[])
252 {
253     if (argc < 2)
254     {
255         throw std::invalid_argument("lc3 [image_file not found]"); // a vm should never
256         » forcefully terminate a host process
257     }
258     for (int j = 1; j < argc; j++)
259     {
260         if (!read_image(argv[j]))
261         {
262             throw std::runtime_error(std::string("Unable to load ") + argv[j]);
263         }
264     }
265     reg[R_COND] = flag_neg;
266     enum
267     {
268         PC_START = 0x3000
269     };
270     reg[R_PC] = PC_START;
271     running_vm = this;
272     std::signal(SIGINT, lc3::handle_interrupt);
273     disable_input_buffering();
274     while (running)
275     {
276         // fetch
277         uint16_t instr = memory_read(reg[R_PC]++);
278         uint16_t op = instr >> 12;
279         if (op_table[op] != nullptr)
280         {
281             op_table[op](this, instr);
282         }
283         else
284         {
285             running = 0;
286         }
287     }
288     restore_input_buffering();
289 }
```

Line by line

Section	Code	What it does and why
Argument check	if (argc < 2) throw std::invalid_argument(...)	A VM without a program is meaningless. The comment explains the choice: <i>a vm should never forcefully terminate a host process</i> , so it throws instead of calling exit().
Load images	for each argv[j]: read_image(...)	A false return (file not found) becomes std::runtime_error naming the file.
Initial flag	reg[R_COND] = flag_neg	The machine needs <i>some</i> defined condition at power-on. The author picks N.
Start address	reg[R_PC] = PC_START (0x3000)	By LC-3 convention user programs start at x3000 (lower memory is reserved for the OS). The local enum keeps the constant scoped to this function.

Section	Code	What it does and why
Register self	<code>running_vm = this</code>	Publishes the object to the global that the signal handler will use.
Install handler	<code>std::signal(SIGINT, lc3::handle_interrupt)</code>	Ctrl-C now restores the terminal instead of leaving it raw.
Raw terminal	<code>disable_input_buffering()</code>	Switch stdin to unbuffered, no-echo mode (Chapter 12).
Loop	<code>while (running) { fetch; decode; dispatch }</code>	The heart of the machine.
Cleanup	<code>restore_input_buffering()</code>	Return the terminal to its original state on normal exit.

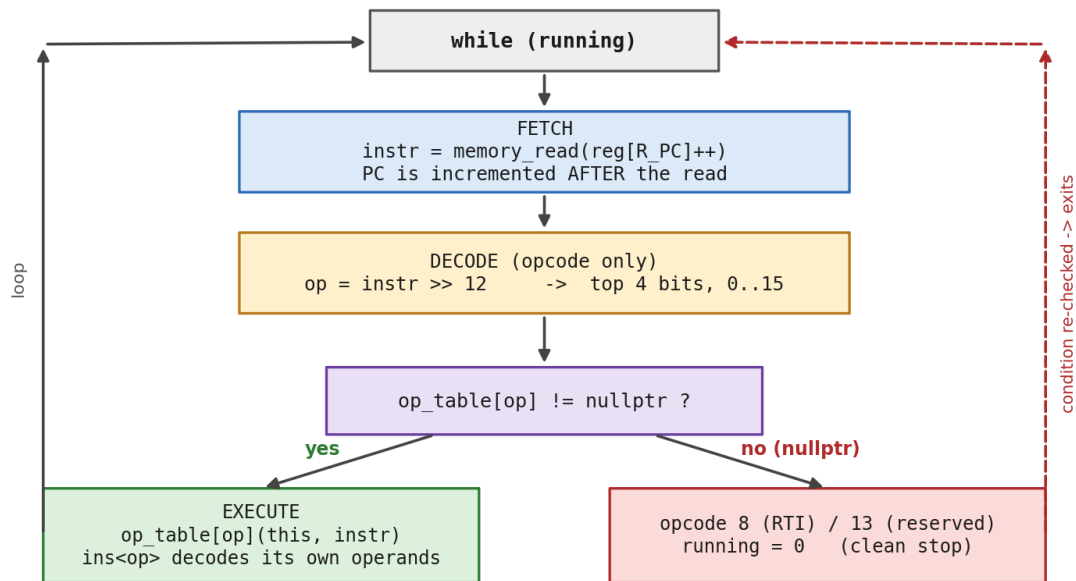


Figure 7.1 — The loop inside `cpu()` (and inside `step()`).

7.2 Fetch, decode, dispatch in three lines

```

uint16_t instr = memory_read(reg[R_PC]++); // fetch + advance PC
uint16_t op    = instr >> 12;              // decode the opcode
if (op_table[op] != nullptr) op_table[op](this, instr); // execute
else running = 0;

```

- **Post-increment.** `reg[R_PC]++` uses the current PC as the address and *then* adds one. By the time the instruction executes, PC already names the *next* instruction. That is precisely the value the LC-3 specification uses for PC-relative addressing, so handlers can add offsets to `reg[R_PC]` without correction. Tests confirm it: a BR at 0x3000 with offset 4 lands on 0x3005.
- **Fetch through `memory_read`.** Instruction fetches use the same accessor as data reads, so the keyboard hooks (Chapter 11) apply uniformly.
- **Decode is one shift.** Because opcodes occupy the top nibble, `instr >> 12` yields 0..15. Operand fields are decoded lazily *inside* each handler.
- **Unknown opcodes stop the machine.** Slots 8 (RTI) and 13 (reserved) contain `nullptr`. Rather than crashing or looping forever, the VM sets `running = 0` and the loop ends.

7.3 step(): the same beat, one at a time

src/lc3.cpp (lines 407-423)

```
407 void lc3::step()
408 {
409     if (!running)
410         return;
411
412     uint16_t instr = memory_read(reg[R_PC]++);
413     uint16_t op = instr >> 12;
414
415     if (op_table[op] != nullptr)
416     {
417         op_table[op](this, instr);
418     }
419     else
420     {
421         running = 0;
422     }
423 }
```

`step()` repeats the loop body once, guarded by `if (!running) return`. It is the hook for debuggers, visualisers, browser front-ends (the header comment says *webassm*) and, crucially, the unit tests: a test writes one instruction to memory, calls `step()`, and inspects the registers.

DESIGN DECISION

Duplicated loop body. `cpu()` and `step()` contain the same three lines. A tidy refactor is to make the loop `while (running) step();`, so there is exactly one place where an instruction is fetched. Duplication is harmless today but any future change (for example an instruction counter or breakpoint check) would have to be made twice.

7.4 Watching a real program run

Consider a small hand-assembled program that counts down from 5, printing each digit. It was run one instruction at a time with `step()`, recording the registers after each step.

Address	Word	Assembly	Effect
0x3000	0x5260	AND R1,R1,#0	R1 = 0
0x3001	0x1265	ADD R1,R1,#5	R1 = 5 (counter)
0x3002	0x1060	ADD R0,R1,#0	loop: R0 = R1 (copy)
0x3003	0x2405	LD R2,+5	R2 = mem[0x3004+5] = mem[0x3009] = '0' (0x30)
0x3004	0x1002	ADD R0,R0,R2	R0 = digit + '0' = ASCII character
0x3005	0xF021	TRAP OUT	print the character in R0
0x3006	0x127F	ADD R1,R1,#-1	counter down
0x3007	0x03FA	BRp -6	if R1 > 0 jump to 0x3008-6 = 0x3002
0x3008	0xF025	TRAP HALT	stop
0x3009	0x0030	.FILL '0'	data word

The first 14 steps (values shown *after* each instruction):

Step	PC	Instruction	R0	R1	R2	COND
0	0x3000	AND R1,R1,#0	0000	0000	0000	Z
1	0x3001	ADD R1,R1,#5	0000	0005	0000	P
2	0x3002	ADD R0,R1,#0	0005	0005	0000	P
3	0x3003	LD R2,+5	0005	0005	0030	P
4	0x3004	ADD R0,R0,R2	0035	0005	0030	P
5	0x3005	TRAP OUT	0035	0005	0030	P

Step	PC	Instruction	R0	R1	R2	COND
6	0x3006	ADD R1,R1,#-1	0035	0004	0030	P
7	0x3007	BRp -6	0035	0004	0030	P
8	0x3002	ADD R0,R1,#0	0004	0004	0030	P
9	0x3003	LD R2,+5	0004	0004	0030	P
10	0x3004	ADD R0,R0,R2	0034	0004	0030	P
11	0x3005	TRAP OUT	0034	0004	0030	P
12	0x3006	ADD R1,R1,#-1	0034	0003	0030	P
13	0x3007	BRp -6	0034	0003	0030	P

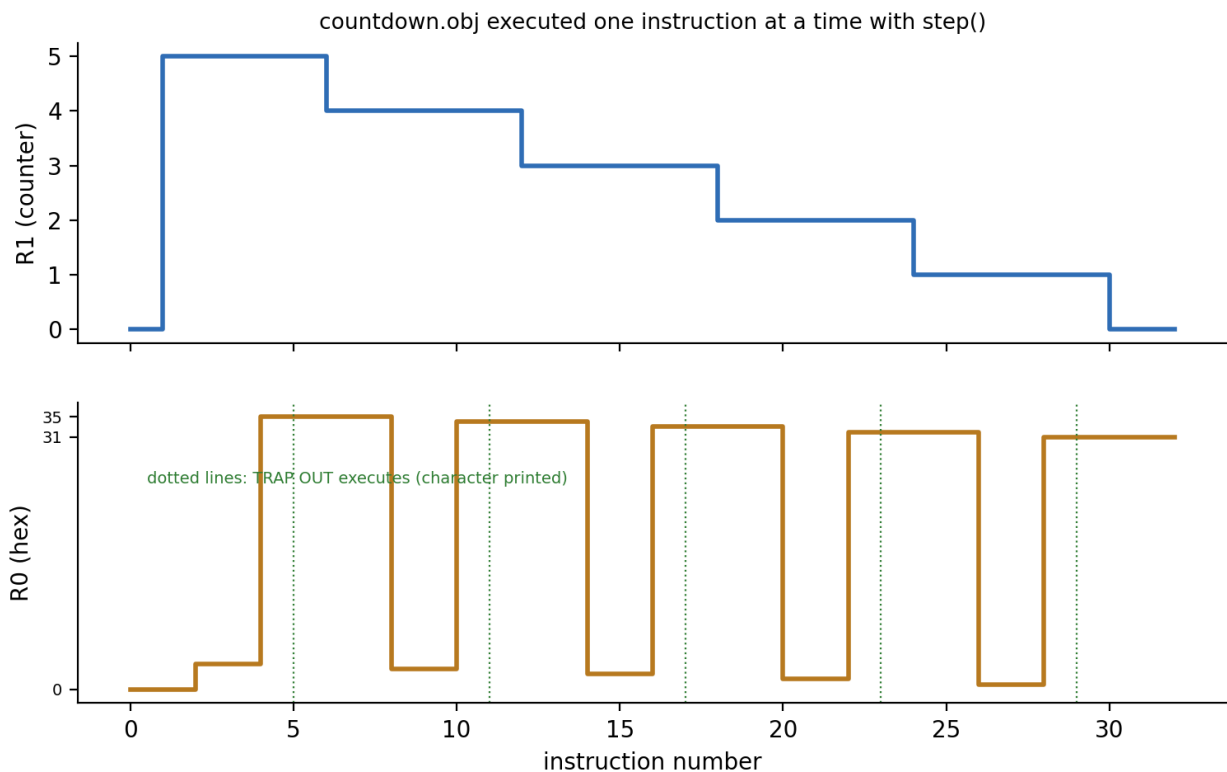


Figure 7.2 — R1 counts 5,4,3,2,1,0; R0 alternates between the counter copy and the ASCII digit that OUT prints.

VERIFIED HERE

Running `./lc3 countdown.obj` prints 54321 followed by the halt banner, exit code 0. Note step 30: when R1 reaches 0 the flag becomes Z, so `BRp` at step 31 falls through to HALT. That is the entire mechanism of a loop on this machine: an ADD sets the flags, a BR reads them.

8 Dispatch by templates and bit masks

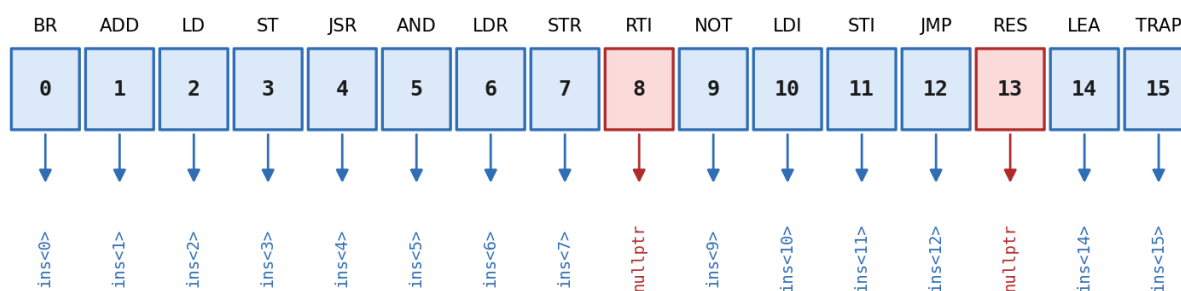
This is the most distinctive idea in the code, and the one worth understanding deeply. The engine needs to run a different piece of logic for each of 16 opcodes. Most emulators use a `switch` statement. This one uses a **table of function pointers** whose entries are all generated from **one** function template, with **bit masks** deciding which parts of that function each opcode keeps.

8.1 The table

src/lc3.cpp (lines 244-249)

```
244 static void (*op_table[16])(lc3 *, uint16_t) =
245     {
246         ins<0>, ins<1>, ins<2>, ins<3>,
247         ins<4>, ins<5>, ins<6>, ins<7>,
248         nullptr, ins<9>, ins<10>, ins<11>,
249         ins<12>, nullptr, ins<14>, ins<15>}; // array that holds pointers to fncs
```

```
static void (*op_table[16])(lc3*, uint16_t)
```



index = instr >> 12 (top nibble). `op_table[op](this, instr)` is one indexed load + one indirect call: O(1). Slots 8 and 13 hold nullptr so no code is generated for them; `cpu()` treats nullptr as 'stop the machine'.

Figure 8.1 — `op_table` maps the opcode straight to a specialised handler.

`void (*op_table[16])(lc3 *, uint16_t)` declares an array of 16 pointers to functions taking `(lc3*, uint16_t)` and returning nothing. The initialiser lists `ins<0> ... ins<15>`: the C++ compiler **instantiates** the template once per listed value. Dispatch is then `op_table[op](this, instr)`: one array lookup and one indirect call, independent of the opcode.

8.2 What a function template is

A template is a recipe for functions. `template <unsigned op> static void ins(lc3*, uint16_t)` is not one function; it is a recipe with a *non-type parameter* `op` that must be a compile-time constant. Writing `ins<1>` asks the compiler to stamp out a complete function in which `op` has been replaced by the literal 1. Because the parameter is known during compilation, anything computed only from it — like `constexpr uint16_t opbit = 1 << op` — is a compile-time constant too.

8.3 The masks: one bit per opcode

Every handler needs to decode some fields (a destination register, a base register, an offset...) but each opcode needs a different subset. Rather than write 15 separate functions, the author wrote one function containing every block, each guarded by `if (MASK & opbit)`. A mask is a 16-bit number whose **bit k is 1 if opcode k wants that block**. For opcode 1 (ADD), `opbit = 1 << 1` is 2, so a block guarded by mask 0x0002 survives; a block guarded by 0x0004 (LD) is dead code in `ins<1>`.

src/lc3.cpp (lines 43-79)

```
43 template <unsigned op>
44 static void ins(lc3 *vm, uint16_t instr)
45 {
46     uint16_t r0, r1, r2, imm5, imm5_flag;
47     uint16_t pc_plus_off, base_plus_off;
48
49     constexpr uint16_t opbit = 1 << op;
50
51     if (0x4EEE & opbit)
52     {
53         r0 = (instr >> 9) & 0x7;
54     }
55     if (0x12F3 & opbit)
56     {
57         r1 = (instr >> 6) & 0x7;
58     }
59     if (0x0022 & opbit)
60     {
61         imm5_flag = (instr >> 5) & 0x1;
62         if (imm5_flag)
63         {
64             imm5 = vm->sign_extend(instr & 0x1F, 5);
65         }
66         else
67         {
68             r2 = instr & 0x7;
69         }
70     }
71
72     if (0x00C0 & opbit)
73     {
74         base_plus_off = vm->reg[r1] + vm->sign_extend(instr & 0x3F, 6);
75     }
76     if (0x4C0D & opbit)
77     {
78         pc_plus_off = vm->reg[lc3::R_PC] + vm->sign_extend(instr & 0x1FF, 9);
79     }
```

Mask	Bits set (opcodes)	Guards this code	Meaning
0x4EEE	ADD, LD, ST, AND, LDR, STR, NOT, LDI, STI, LEA	<code>r0 = (instr >> 9) & 7</code>	destination or source register in bits 11-9
0x12F3	BR, ADD, JSR, AND, LDR, STR, NOT, JMP	<code>r1 = (instr >> 6) & 7</code>	first source / base register in bits 8-6
0x0022	ADD, AND	<code>imm5_flag</code> , then <code>imm5</code> or <code>r2</code>	the ADD/AND mode bit (bit 5) and the second operand
0x00C0	LDR, STR	<code>base_plus_off = reg[r1] + sext(off6)</code>	LDR/STR effective address
0x4C0D	BR, LD, ST, LDI, STI, LEA	<code>pc_plus_off = PC + sext(off9)</code>	PC-relative effective address
0x4666	ADD, LD, AND, LDR, NOT, LDI, LEA	<code>update_flags(r0)</code>	instructions that write a general register and set N/Z/P

		BR	ADD	LD	ST	JSR	AND	LDR	STR	RTI	NOT	LDI	STI	JMP	RES	LEA	TRAP
0x4EEE	r0 = DR/SR [11:9]		x	x	x		x	x	x		x	x	x			x	
0x12F3	r1 = SR1/BaseR [8:6]	x	x			x	x	x	x		x			x			
0x0022	imm5 flag, r2, imm5		x				x										
0x00C0	base_plus_off							x	x								
0x4C0D	pc_plus_off	x		x	x							x	x			x	
0x4666	update_flags(r0)		x	x			x	x			x	x				x	
0x0001	BR body	x															
0x0002	ADD body		x														
0x0004	LD body			x													
0x0008	ST body				x												
0x0010	JSR body					x											
0x0020	AND body						x										
0x0040	LDR body							x									
0x0080	STR body								x								
0x0200	NOT body										x						
0x0400	LDI body											x					
0x0800	STI body												x				
0x1000	JMP body													x			
0x4000	LEA body															x	
0x8000	TRAP body																x

Columns are opcodes (bit position = opcode number). A set mask bit means that opcode's ins<op> keeps that block of code.

Figure 8.2 — All twenty masks as a matrix. Amber rows decode operands; green rows are the per-opcode bodies (one bit each).

VERIFIED HERE

Every mask cross-checked against the ISA. Each instruction format was encoded independently (which opcodes have a DR/SR in bits 11-9, which have a base register in bits 8-6, which take a PCOffset9, and so on) and compared with the six operand masks. All required bits are present. Extra bits: **0x12F3 also includes BR.** BR also decodes bits 8-6 into **r1**, which it never uses, so this is harmless. The six masks are correct.

8.4 Why dead code disappears

For ins<1>, every if (mask & opbit) test compares two constants, for example 0x4EEE & 2. The compiler evaluates such tests while compiling and deletes the blocks that can never run. What remains of ins<1> is essentially:

```
// ins<1> (ADD) after the compiler folds the constant conditions
uint16_t r0 = (instr >> 9) & 7;
uint16_t r1 = (instr >> 6) & 7;
uint16_t imm5_flag = (instr >> 5) & 1;
if (imm5_flag) imm5 = sign_extend(instr & 0x1F, 5); else r2 = instr & 7;
reg[r0] = imm5_flag ? reg[r1] + imm5 : reg[r1] + reg[r2];
update_flags(r0);
```

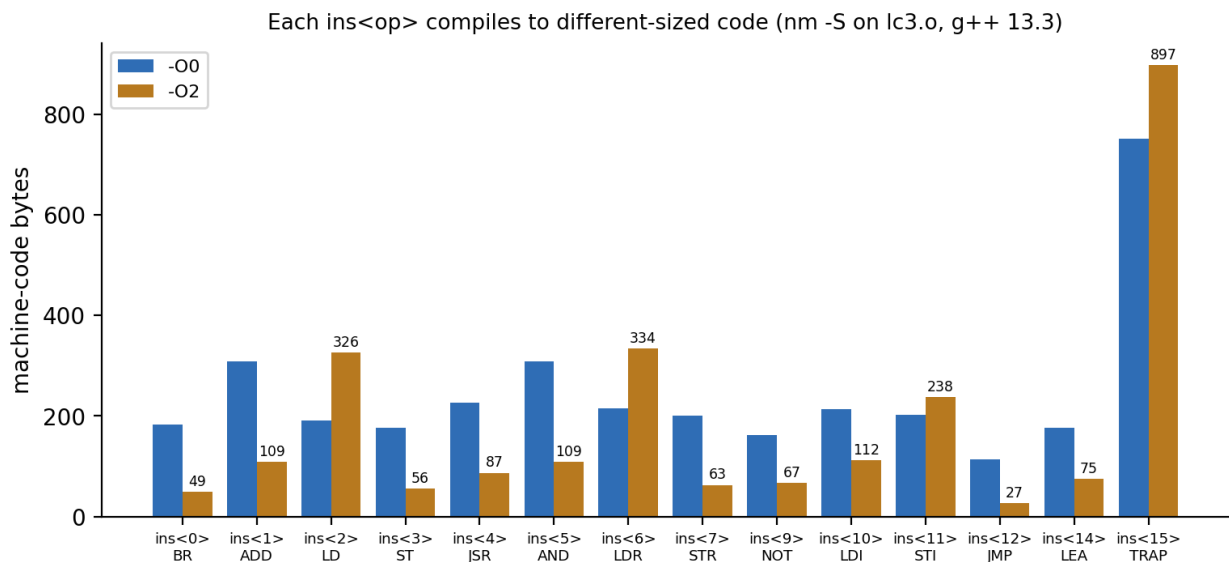


Figure 8.3 — Machine-code size of each instantiation. Small handlers stay small; TRAP, which carries the whole system-call switch, is by far the largest.

VERIFIED HERE

Evidence from the compiled object. `nm -S --size-sort` on `lc3.o` shows the instantiations have very different sizes. At `-O2`: `JMP` 27 bytes, `BR` 49, `ST` 56, `TRAP` 897. Even at `-O0` (no optimisation) GCC already removes constant-false branches: `ins<0>` is 183 bytes while `ins<15>` is 751. At `-O2` `LD` (326), `LDR` (334) and `STI` (238) are larger than `LDI` (112), which is most likely GCC inlining `memory_read` at some call sites and calling it out of line at others; the assembly was not inspected further, so treat that explanation as probable rather than proven.

8.5 A subtle point: uninitialised locals

The prologue declares `r0`, `r1`, `r2`, `imm5`, `imm5_flag`, `pc_plus_off`, `base_plus_off` with no initial value. A variable is written only in blocks whose mask includes the opcode, and read only in blocks with the *same* or a *subset* mask, so no instantiation ever reads an unset value. Because each block's guard is a constant, the compiler can prove it too: building with `g++ -O2 -Wall -Wextra` produced **zero** maybe-uninitialised warnings. This invariant is not enforced by the language, though — it is enforced by the masks being consistent, which is why cross-checking them (above) matters.

8.6 Alternatives and trade-offs

Approach	Strengths	Costs
Classic switch(op) with 15 case bodies	Most familiar; each opcode's logic is together; a dense switch usually compiles to a jump table too, which is also $O(1)$.	Operand decoding is repeated in many cases (or hoisted and computed unnecessarily).
This design: one template, masks	Operand decoding is written once; each instantiation contains only what it needs; adding a register field is a one-line change.	Logic for one opcode is scattered across several guarded blocks; correctness depends on masks staying consistent; unfamiliar to many readers.
Template + if constexpr (C++17)	Same idea, but the language <i>guarantees</i> the dead branches are discarded before code generation.	Needs C++17 (the CMake file already requests it).

DESIGN DECISION

What the README claim really buys. The README says the table "bypasses standard switch statement bottlenecks". A dense switch on a 4-bit value is normally compiled to an indirect jump through a table as well, so both approaches are constant-time. The real benefit of the author's design is the *specialisation*: each opcode gets a handler with only its own decoding work, and the mask trick removes repeated code. Whether it is measurably faster than a switch is an empirical question (a simple benchmark running a long program under both would answer it).

NOTE

Because the parameter is *unsigned op* and the guard is a runtime-style `if`, the compiler still has to *parse and type-check* every block in every instantiation. That is why the compiler printed 29 warnings about an ignored `read()` result: the TRAP code is textually present in all 15 instantiations even though only `ins<15>` keeps it. Using `if constexpr` would silence that noise.

9 Every instruction, explained

Each section below gives the encoding, the semantics, the code, and real results. All 'after' values were produced by executing the encoded instruction in the actual VM, starting from a fresh machine with PC = 0x3000.

9.1 Extracting the fields

src/lc3.cpp (lines 43-79)

```

43  template <unsigned op>
44  static void ins(lc3 *vm, uint16_t instr)
45  {
46      uint16_t r0, r1, r2, imm5, imm5_flag;
47      uint16_t pc_plus_off, base_plus_off;
48
49      constexpr uint16_t opbit = 1 << op;
50
51      if (0x4EEE & opbit)
52      {
53          r0 = (instr >> 9) & 0x7;
54      }
55      if (0x12F3 & opbit)
56      {
57          r1 = (instr >> 6) & 0x7;
58      }
59      if (0x0022 & opbit)
60      {
61          imm5_flag = (instr >> 5) & 0x1;
62
63          if (imm5_flag)
64          {
65              imm5 = vm->sign_extend(instr & 0x1F, 5);
66          }
67          else
68          {
69              r2 = instr & 0x7;
70          }
71      }
72      if (0x00C0 & opbit)
73      {
74          base_plus_off = vm->reg[r1] + vm->sign_extend(instr & 0x3F, 6);
75      }
76      if (0x4C0D & opbit)
77      {
78          pc_plus_off = vm->reg[lc3::R_PC] + vm->sign_extend(instr & 0x1FF, 9);
79      }

```

The prologue is the decoder. Reading it with Figure 2.3 open, each block is a direct translation of a bit-field diagram: shift the wanted field down to bit 0, then AND with a mask of ones as wide as the field (0x7 = 3 bits, 0x1F = 5 bits, 0x3F = 6 bits, 0x1FF = 9 bits). Note the two-step treatment of ADD/AND: first read the mode bit (bit 5); only then decide whether the last operand is a register or an immediate.

9.2 Operate instructions: ADD, AND, NOT

src/lc3.cpp (lines 90-117)

```

90      if (0x0002 & opbit) // ADD
91      {
92          if (imm5_flag)
93          {
94              vm->reg[r0] = vm->reg[r1] + imm5;
95          }
96          else
97          {
98              vm->reg[r0] = vm->reg[r1] + vm->reg[r2];
99          }
100     }
101
102     if (0x0020 & opbit) // AND
103     {
104         if (imm5_flag)
105         {

```

```

106         vm->reg[r0] = vm->reg[r1] & imm5;
107     }
108     else
109     {
110         vm->reg[r0] = vm->reg[r1] & vm->reg[r2];
111     }
112 }
113
114 if (0x0200 & opbit) // Not
115 {
116     vm->reg[r0] = ~vm->reg[r1];
117 }

```

ADD and AND have two forms selected by bit 5. Register mode adds (or ANDs) two registers; immediate mode uses a 5-bit signed constant, sign-extended, giving a range of -16..+15. The result goes into `reg[r0]` and the flags are updated at the very end of the function (mask 0x4666). **NOT** inverts every bit of one register. The compiler's `~vm->reg[r1]` promotes to int, but assignment back to a 16-bit register truncates, so the upper bits vanish.

CONCEPT

NOT plus ADD gives subtraction. Two's complement negation is 'invert then add 1'. The LC-3 has no SUB, so a program computes $A - B$ as NOT B; ADD B,B,#1; ADD R,A,B. This is why NOT exists and why NOT encodes its unused bits as 111111 (Figure 2.3).

Instruction	Encoding	Before	After (run in the real VM)
ADD reg	0x1042	R1=0005 R2=0003	R0=0008 COND=1 PC=3001
ADD imm	0x107D	R1=000A	R0=0007 COND=1
ADD overflow	0x1261	R1=7FFF	R1=8000 COND=4
AND reg	0x5042	R1=0F0F R2=00FF	R0=000F COND=1
AND imm	0x507F	R1=1234	R0=1234 COND=1
NOT	0x907F	R1=00FF	R0=FF00 COND=4

9.3 Data movement: LD, LDI, LDR, LEA, ST, STI, STR

src/lc3.cpp (lines 141-168)

```

141     if (0x0004 & opbit)
142     {
143         vm->reg[r0] = vm->memory_read(pc_plus_off);
144     } // Ld
145     if (0x0400 & opbit)
146     {
147         vm->reg[r0] = vm->memory_read(vm->memory_read(pc_plus_off));
148     } // ldi
149     if (0x0040 & opbit)
150     {
151         vm->reg[r0] = vm->memory_read(base_plus_off);
152     } // ldr
153     if (0x4000 & opbit)
154     {
155         vm->reg[r0] = pc_plus_off;
156     } // lea
157     if (0x0008 & opbit)
158     {
159         vm->memory_write(pc_plus_off, vm->reg[r0]);
160     } // st
161     if (0x0800 & opbit)
162     {
163         vm->memory_write(vm->memory_read(pc_plus_off), vm->reg[r0]);
164     } // sti
165     if (0x0080 & opbit)
166     {
167         vm->memory_write(base_plus_off, vm->reg[r0]);
168     } // str

```

Seven instructions, seven one-line bodies — all the hard work (computing `pc_plus_off` and `base_plus_off`) was done in the prologue. The differences are just the number of memory hops:

Instruction	Hops	Code	Use case
LD	1	reg[r0] = memory_read(pc_plus_off)	load a nearby variable
LDI	2	reg[r0] = memory_read(memory_read(pc_plus_off))	follow a pointer stored nearby (reach any address)
LDR	1	reg[r0] = memory_read(base_plus_off)	arrays, struct fields, stack frames
LEA	0	reg[r0] = pc_plus_off	get an address, e.g. of a string, without reading memory
ST	1	memory_write(pc_plus_off, reg[r0])	store to a nearby variable
STI	2	memory_write(memory_read(pc_plus_off), reg[r0])	store through a pointer
STR	1	memory_write(base_plus_off, reg[r0])	store into an array element

Loads go through `memory_read`, so a load from the keyboard addresses triggers the device logic. Stores use `memory_write`, a plain array write. Loads set the flags (mask 0x4666 includes LD, LDR, LDI, LEA); stores do not.

Instruction	Encoding	Before	After (run in the real VM)
LD	0x2005	M[3006]=9999	R0=9999 COND=4
LDI	0xA002	M[3003]=4000 M[4000]=BEEF	R0=BEEF COND=4
LDR	0x6045	R1=4000 M[4005]=8888	R0=8888 COND=4
LEA	0xE005	COND=N	R0=3006 COND=1
ST	0x3005	R0=AAAA	M[3006]=AAAA
STI	0xB002	R0=1111 M[3003]=4000	M[4000]=1111
STR	0x7045	R0=2222 R1=4000	M[4005]=2222

CONCEPT

LEA is not a load. LEA (load effective address) never touches memory; it computes PC + offset and puts the *address* in a register. That is how a program obtains a pointer to its own string before calling PUTS: `LEA R0, msg ; PUTS`. The verified example above shows R0 = 0x3006 for LEA at 0x3000 with offset +5 (0x3001 + 5).

9.4 Control flow: BR, JMP/RET, JSR/JSRR

src/lc3.cpp (lines 81-88)

```

81     if (0x0001 & opbit) // BR
82     {
83         uint16_t cond = (instr >> 9) & 0x7;
84         if (cond & vm->reg[lc3::R_COND])
85         {
86             vm->reg[lc3::R_PC] = pc_plus_off;
87         }
88     }

```

BR extracts the three condition bits and ANDs them with `reg[R_COND]`. If any requested condition is currently true, PC is replaced by the pre-computed PC + offset. With all three bits set (BRnzp, encoded 0x0E00 | offset) the branch is unconditional; with none set it is a no-op. **Branches never change the flags.**

src/lc3.cpp (lines 119-139)

```

119    if (0x1000 & opbit) // JMP
120    {
121        vm->reg[lc3::R_PC] = vm->reg[r1];
122    }
123
124    if (0x0010 & opbit) // JSR
125    {
126        uint16_t long_flag = ((instr >> 11) & 1);
127        vm->reg[lc3::R_R7] = vm->reg[lc3::R_PC];

```

```

128
129     if (long_flag)
130     {
131         pc_plus_off = vm->reg[lc3::R_PC] + vm->sign_extend(instr & 0x7FF, 11);
132         vm->reg[lc3::R_PC] = pc_plus_off;
133     }
134
135     else
136     {
137         vm->reg[lc3::R_PC] = vm->reg[r1];
138     }
139 }

```

JMP copies a register into PC (an absolute jump). The famous **RET** is just **JMP R7**, encoded 0xC1C0. **JSR** (jump to subroutine) first saves the return address, the already-incremented PC, into R7 — the link register — then jumps. Bit 11 selects the form: 1 = JSR with an 11-bit PC-relative offset (range about ± 1024 words); 0 = JSRR, jump to the address in a base register.

WATCH OUT

JSRR R7 jumps to the wrong place. The code writes `R7 = PC` before reading `reg[r1]`. For every base register except R7 that order is harmless, but **JSRR R7** reads R7 after it has been overwritten with the return address. *Reproduced:* with R7 = 0x5000, executing 0x41C0 (JSRR R7) at 0x3000 left PC = 0x3001 (should be 0x5000). The specification saves the old PC in a temporary, reads the base register, then writes R7. Fix: read the target first, e.g. `uint16_t target = reg[r1]; reg[R7] = PC; PC = target;`.

Instruction	Encoding	Before	After (run in the real VM)
BRz taken	0x0404	COND=Z	PC=3005
BRz not taken	0x0404	COND=N	PC=3001
BRnzp #-2	0x0FFE	COND=P	PC=2FFF
JMP R1	0xC040	R1=5000	PC=5000
RET (JMP R7)	0xC1C0	R7=1234	PC=1234
JSR	0x480A	-	R7=3001 PC=300B
JSRR R2	0x4080	R2=4000	R7=3001 PC=4000

Note the row *BRnzp #-2*: from PC = 0x3001 the offset -2 gives 0x2FFF, demonstrating sign extension of the 9-bit offset and that branches can go backwards — the basis of every loop.

9.5 What is not implemented: RTI and opcode 13

RTI (return from interrupt) belongs to the LC-3's privileged mode and interrupt machinery, which this VM does not model. Opcode 13 is unused in the ISA. Both slots hold `nullptr`, so executing either simply stops the machine cleanly.

Instruction	Encoding	Before	After (run in the real VM)
RTI (op 8)	0x8000	-	running=0 PC=3001
reserved (op 13)	0xD000	-	running=0

9.6 The tail: updating the flags

src/lc3.cpp (lines 237-242)

```

237     if (0x4666 & opbit)
238     {
239         {
240             vm->update_flags(r0);
241         }
242     }

```

One shared epilogue handles condition codes for all seven flag-setting instructions. Because it runs *after* the operation, it sees the newly written register value. It is a neat consequence of the mask design: 'sets

flags' is stated once, as a mask, not repeated in seven bodies.

10 System calls: the TRAP routines

A LC-3 program cannot talk to the screen or keyboard directly (in the reference system it calls operating-system routines through the TRAP instruction). A TRAP carries an 8-bit **trap vector** in its low byte. On real LC-3 hardware the CPU looks the vector up in a table in low memory and jumps to OS code; here the OS does not exist, so the VM performs each service in host C++.

Vector	Name	Behaviour in this VM
0x20	GETC	Read one key (blocking) into R0, no echo; set flags.
0x21	OUT	Print the character in R0 (low 8 bits).
0x22	PUTS	Print the zero-terminated string starting at address R0, one character per 16-bit word.
0x23	IN	Print the prompt "Enter a character: ", read one key, echo it, R0 = key, set flags.
0x24	PUTSP	Print a zero-terminated string with two characters packed per word (low byte first).
0x25	HALT	Print "-- HALT EXECUTED --" and set running = 0.

src/lc3.cpp (lines 169-235)

```

169     if (0x8000 & opbit) // trap
170     {
171         vm->reg[lc3::R_R7] = vm->reg[lc3::R_PC];
172
173         switch (instr & 0xFF)
174         {
175             case lc3::TRAP_GETC:
176             {
177                 char c = 0;
178                 read(STDIN_FILENO, &c, 1); // read directly from os kernel's buffer replacment for
179                 » cin(idk error)
180                 vm->reg[lc3::R_R0] = static_cast<uint16_t>(c);
181                 vm->update_flags(lc3::R_R0);
182                 break;
183             }
184             case lc3::TRAP_OUT:
185             {
186                 std::cout << static_cast<char>(vm->reg[lc3::R_R0] & 0xFF) << std::flush;
187                 break;
188             }
189             case lc3::TRAP_PUTS:
190             {
191                 uint16_t addr = vm->reg[lc3::R_R0];
192                 while (addr < MEMORY_SIZE && vm->memory[addr] != 0)
193                 {
194                     std::cout << static_cast<char>(vm->memory[addr]);
195                     addr++;
196                 }
197                 std::cout << std::flush;
198                 break;
199             }
200             case lc3::TRAP_IN:
201             {
202                 std::cout << "Enter a character: " << std::flush;
203                 char c = 0;
204                 read(STDIN_FILENO, &c, 1);
205                 std::cout << c << std::flush;
206                 vm->reg[lc3::R_R0] = static_cast<uint16_t>(c);
207                 vm->update_flags(lc3::R_R0);
208                 break;
209             }
210             case lc3::TRAP_PUTSP:
211             {
212                 uint16_t addr = vm->reg[lc3::R_R0];
213                 while (addr < MEMORY_SIZE && vm->memory[addr] != 0)
214                 {
215                     char c1 = (vm->memory[addr]) & 0xFF;
216                     std::cout << c1;
217                     char c2 = (vm->memory[addr]) >> 8;
218                     if (c2)
219                         std::cout << c2;
220                     addr++;
221                 }
222             }
223         }
224     }

```

```

221         }
222         std::cout << std::flush;
223         break;
224     }
225     case lc3::TRAP_HALT:
226     {
227         std::cout << "\n-- HALT EXECUTED --\n"
228                 << std::flush;
229         vm->running = 0;
230         break;
231     }
232     default:
233         break;
234 }
235 }

```

10.1 The common prologue

R7 = PC stores the return address exactly as real hardware would, even though the routine returns immediately and nothing reads it. It preserves the architectural contract: after a TRAP, R7 holds the address following the TRAP. The dispatch is a **switch** on the low byte **instr & 0xFF**; the trap vectors are the enum from Chapter 4.

10.2 Input traps and why read() replaces cin

GETC and IN call the POSIX **read(STDIN_FILENO, &c, 1)** directly on file descriptor 0. The author's comment records the reason: *read directly from os kernel's buffer, replacement for cin*. **std::cin** has its own buffering and line handling, and would not interoperate with the raw terminal mode set in Chapter 12; **read** returns as soon as one byte is available. GETC then stores the character in R0 and sets the flags; IN additionally prints a prompt and echoes the key (the terminal's own echo is off).

VERIFIED HERE

Run under a pseudo-terminal. A 3-word program (GETC, OUT, HALT) was started under a pty and the key 'z' was typed. Output: 'z\r\n-- HALT EXECUTED --\r\n'. Only **one** 'z' appears — GETC does not echo and the VM's raw mode turned the terminal's own echo off; the visible 'z' comes from OUT. The IN program produced 'Enter a character: k\r\n-- HALT EXECUTED --\r\n': prompt, then the echoed key.

10.3 Output traps

OUT prints **static_cast<char>(reg[R0] & 0xFF)**: masking with 0xFF keeps only the low byte, since a character is 8 bits inside a 16-bit register. Every output call ends with **std::flush** so text appears immediately rather than waiting in the stream buffer; without it a prompt could sit unseen until the program ended.

PUTS walks memory from the address in R0, printing the low byte of each word until it meets a zero word. The LC-3 stores strings one character per word, which wastes half the space but keeps loads simple.

PUTSP is the packed variant: the low byte holds the first character and the high byte the second; a zero high byte marks an odd-length string's end, and a zero word ends the string.

Instruction	Encoding	Before	After (run in the real VM)
TRAP OUT	0xF021	R0=0041	stdout="A" R7=3001
TRAP PUTS	0xF022	M[4000..]=Hi!	stdout="Hi!"
TRAP PUTSP	0xF024	M[4000]=6948 M[4001]=0021	stdout="Hi!"
HALT	0xF025	-	running=0

10.4 HALT and unknown traps

HALT prints a banner and sets `running = 0`; the run loop ends and the terminal is restored. An unknown trap vector (anything outside 0x20-0x25) falls into `default: break`; and is silently ignored: execution continues at the next instruction.

Instruction	Encoding	Before	After (run in the real VM)
TRAP x00FF unknown	0xF0FF	-	running=1 PC=3001 R7=3001

10.5 Things to know about the trap code

WATCH OUT

Characters above 0x7F. In GETC/IN the key is stored in a plain `char` (signed on most platforms) and then cast to `uint16_t`. A byte such as 0xE9 becomes 0xFFE9, not 0x00E9. *Verified:* `static_cast<uint16_t>((char)0xE9)` evaluates to 0xFFE9. ASCII input is unaffected. Fix: read into `unsigned char`.

WATCH OUT

The PUTS bounds check cannot fail. The loop condition `addr < MEMORY_SIZE` compares a `uint16_t` (maximum 65,535) with 65,536, so it is always true. The real protection is that `addr++` wraps to 0 after 0xFFFF; a string with no terminator would keep printing around the address space until a zero word appears. Practical impact is nil for valid programs; the check is simply redundant (the type already bounds the index).

WATCH OUT

read() results are ignored. The compiler issued 29 `-Wunused-result` warnings. If stdin is closed or at end of file, `read` returns 0 and the character stays 0, so GETC returns 0 forever. A robust version checks the return value and decides what end-of-input should mean (for example halting).

11 Memory-mapped keyboard input

11.1 The idea: devices that look like memory

The CPU has only two ways to talk to the world: memory addresses and instructions. A classic answer is **memory-mapped I/O**: certain addresses do not refer to storage but to a *device register*. Reading such an address asks the device a question; writing sends it a command. The programmer uses ordinary LD/ST instructions and needs no special I/O opcodes.

The LC-3 keyboard has two registers. **KBSR** (status, 0xFE00): bit 15 is 1 when a key has been typed and is waiting. **KBDR** (data, 0xFE02): the low byte holds the key.

11.2 memory_read: the device hook

src/lc3.cpp (lines 340-368)

```
340 uint16_t lc3::memory_read(uint16_t address)
341 {
342     if (address == MR_KBSR)
343     {
344         if (check_key())
345         {
346             memory[MR_KBSR] = 1 << 15; // Set ready bit, but DO NOT consume key yet
347         }
348         else
349         {
350             memory[MR_KBSR] = 0;
351         }
352     }
353     else if (address == MR_KBDR)
354     {
355         if (check_key())
356         {
357             memory[MR_KBSR] = 0;
358             char c = 0;
359             read(STDIN_FILENO, &c, 1); // Reads directly from the POSIX file descriptor
360             memory[MR_KBDR] = static_cast<uint16_t>(c);
361         }
362         else
363         {
364             memory[MR_KBDR] = 0;
365         }
366     }
367     return memory[address];
368 }
```

src/lc3.cpp (lines 335-338)

```
335 void lc3::memory_write(uint16_t address, uint16_t val)
336 {
337     memory[address] = val;
338 }
```

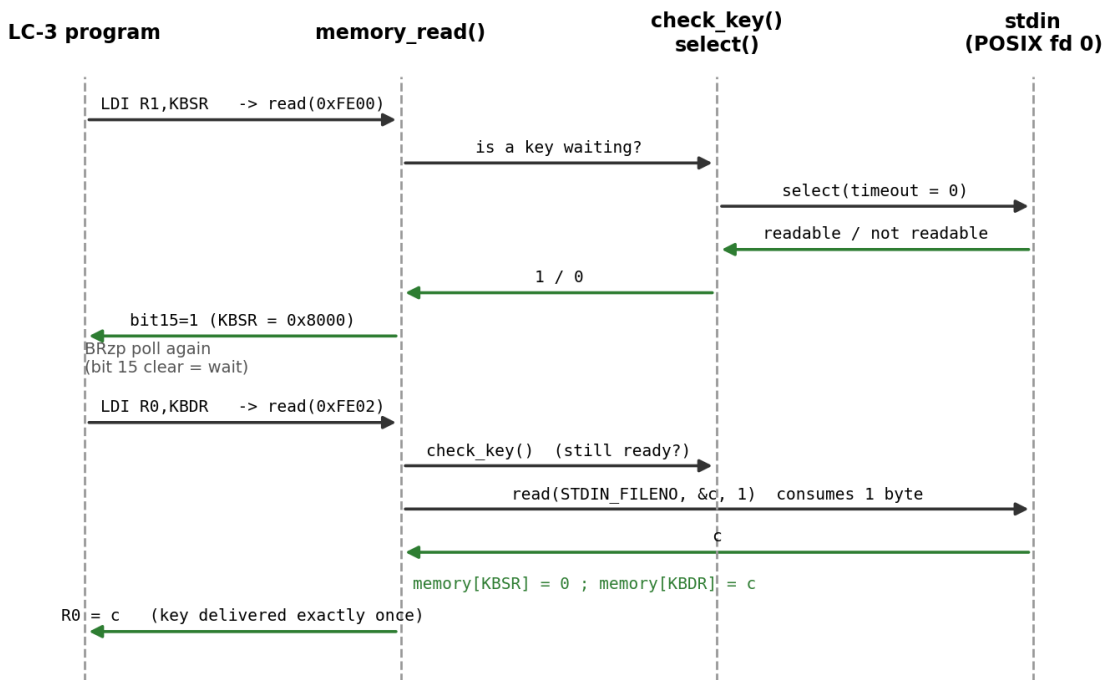


Figure 11.1 — One polling loop iteration and one data read, as they travel down through the layers.

Reading the code

- **Status read (KBSR).** The function asks `check_key()` whether a key is pending. If so it stores `1 << 15` (bit 15) in `memory[MR_KBSR]`; otherwise 0. It then returns `memory[address]` as usual. Crucially it does *not* read the key: checking status must not consume input.
- **Data read (KBDR).** If a key is pending it clears the ready bit, reads one byte with `read()`, and stores it in KBDR. If no key is pending KBDR reads as 0. Reading data *consumes* the key, exactly like a real hardware register that empties when read.
- **Writes are plain.** `memory_write` has no hooks: writing to 0xFE00 just stores the value. That is fine for the keyboard (its only writable bit is the interrupt-enable bit, which is not implemented).

11.3 check_key: asking without waiting

src/lc3.cpp (lines 385-395)

```

385 uint16_t lc3::check_key()
386 {
387     fd_set readfds;
388     FD_ZERO(&readfds);
389     FD_SET(STDIN_FILENO, &readfds);
390
391     struct timeval timeout;
392     timeout.tv_sec = 0;
393     timeout.tv_usec = 0;
394     return select(STDIN_FILENO + 1, &readfds, nullptr, nullptr, &timeout) > 0;
395 }
  
```

`select()` is the POSIX way to ask 'is any of these file descriptors ready?'. The code builds a set containing only descriptor 0 (`FD_ZERO`, `FD_SET`), sets the timeout to zero seconds and zero microseconds (so it **polls** rather than waits), and returns true if at least one descriptor is readable. The first argument `STDIN_FILENO + 1` is 'highest descriptor number + 1', which POSIX requires.

CONCEPT

Why polling needs a zero timeout. A blocking read would freeze the whole VM whenever a program merely *asks* whether a key is ready, so games could not animate while waiting for input. With `select()` and a zero timeout, the VM keeps running and the program decides how long to wait.

11.4 The polling idiom in LC-3 code

```
POLL    LDI    R1, KBSRPTR    ; R1 = mem[ mem[KBSRPTR] ] = the status register
        BRzp  POLL           ; bit 15 clear -> value >= 0 -> nothing typed yet, ask again
        LDI    R0, KBD RPTR    ; a key is ready: read it (this consumes it)
        ...
KBSRPTR .FILL  xFE00
KBD RPTR .FILL  xFE02
```

The trick is that bit 15 is also the *sign* bit. When a key is waiting, KBSR reads as 0x8000, which is negative, so the condition code becomes N and BRzp is not taken. LDI is used because the addresses 0xFE00/0xFE02 are too far away for a PC-relative LD; the pointer words live next to the code.

VERIFIED HERE

This exact loop was run. The program above (0xA205, 0x07FE, 0xA004, TRAP OUT, HALT) was started under a pseudo-terminal. It spun in the poll loop until the key 'q' was sent, then printed it. Output captured: 'q\r\n-- HALT EXECUTED --\r\n', exit code 0. The keyboard registers work end-to-end.

DESIGN DECISION

No display registers. Programs that print through the display status/data registers (0xFE04/0xFE06) would not work, because those addresses are ordinary RAM in this VM. Output is provided only through TRAP routines. That covers the common assembled programs, and it is a defensible scope choice. Supporting DSR/DDR would mean adding hooks in `memory_write` for those addresses.

NOTE

Reasoned, not executed: if stdin reaches end-of-file, select() reports it readable forever and read() returns 0, so the VM would report 'key ready' with data 0 on every poll. Programs that poll in a loop would spin. Ending the run on EOF would be a natural refinement.

PART V — THE OUTSIDE WORLD

12 The terminal, signals, and shutdown

12.1 Why the terminal must be changed

By default a terminal is in **canonical (cooked) mode**: the kernel collects a whole line, lets you edit it with Backspace, echoes each character, and delivers the line only when you press Enter. A game needs the opposite — each key delivered the instant it is pressed, and no automatic echo. The POSIX interface for this is **termios**.

src/lc3.cpp (lines 370-378)

```
370 void lc3::disable_input_buffering()
371 {
372     tcgetattr(STDIN_FILENO, &original_tio);
373     struct termios new_tio = original_tio;
374     new_tio.c_lflag &= ~(ICANON | ECHO);
375     new_tio.c_cc[VMIN] = 1; // Guarantee blocking until 1 char is typed
376     new_tio.c_cc[VTIME] = 0; // No timeout
377     tcsetattr(STDIN_FILENO, TCSANOW, &new_tio);
378 }
```

src/lc3.cpp (lines 380-383)

```
380 void lc3::restore_input_buffering()
381 {
382     tcsetattr(STDIN_FILENO, TCSANOW, &original_tio);
383 }
```

Statement	Meaning
tcgetattr(STDIN_FILENO, &original_tio)	Save the current terminal settings into the member original_tio so they can be restored exactly.
new_tio = original_tio	Start from the existing settings and change only what is needed.
new_tio.c_lflag &= ~(ICANON ECHO)	Clear two local-mode flags: ICANON (line buffering) and ECHO (automatic echo). Every other flag, including ISIG (Ctrl-C generates SIGINT), is untouched.
c_cc[VMIN] = 1	A read() returns as soon as at least 1 byte is available (blocks until then).
c_cc[VTIME] = 0	No timeout: wait indefinitely for that byte.
tcsetattr(..., TCSANOW, &new_tio)	Apply immediately.

DESIGN DECISION

Modify a copy, restore the original. Copying the existing settings and flipping two bits is far safer than building a termios from scratch, which might silently clear flags the user's terminal needs. Saving the original in a *member* (not a global) ties the saved state to the machine that changed it.

VERIFIED HERE

Raw mode really is on. Under a pseudo-terminal, typing 'z' produced a single 'z' in the output (from TRAP OUT). If the terminal's ECHO had still been enabled, the typed key would have been echoed by the terminal as well and 'zz' would appear.

12.2 Ctrl-C and the signal handler

src/lc3.cpp (lines 397-405)

```
397 void lc3::handle_interrupt(int signal)
398 {
399     if (running_vm)
400     {
401         running_vm->restore_input_buffering();
402         running_vm->running = 0;
403     }
404     std::printf("\n-- Execution Interrupted --\n");
405 }
```

A **signal** is an asynchronous notification from the operating system. Pressing Ctrl-C in a terminal that still has ISIG enabled makes the kernel send SIGINT to the foreground process; by default that kills the process instantly — *leaving the terminal in raw mode*, so the user's shell would stop echoing typed characters. The handler prevents that by restoring the terminal first and asking the loop to stop.

- `std::signal(SIGINT, lc3::handle_interrupt)` registers the handler. It runs on the process's normal call stack whenever the signal arrives, interrupting the VM loop between machine instructions.
- `running_vm` is a global pointer to the running machine, set in `cpu()`. The handler is a static member function — it has no `this` — so it needs a global to find the machine whose terminal it must restore.
- Setting `running = 0` makes the `while (running)` condition false; the loop exits and `cpu()` returns normally.

VERIFIED HERE

SIGINT test. A program consisting of a single instruction that branches to itself (`BRnzp #-1, 0x0FFF`) was started, and SIGINT was sent after 0.5 s. The process printed '`\n-- Execution Interrupted --\n`' and exited with code 0 after 0.5 s. The interrupt handling works, and the VM shuts down through its normal path.

12.3 Hardening opportunities in the signal path

Observation	Why it matters	Suggested fix
The handler calls printf	Only a small set of functions are async-signal-safe; printf is not. Works in practice here, but it is technically undefined if the signal lands during another stdio call.	Use <code>write(STDOUT_FILENO, ...)</code> with a fixed string.
running is a plain int	A variable shared with a signal handler should be volatile <code>std::sig_atomic_t</code> (or <code>std::atomic</code>) so the compiler cannot cache it and the write is atomic.	Change the type of <code>running</code> .
running_vm is never cleared	If the <code>lc3</code> object were destroyed and a later SIGINT arrived, the handler would dereference a dangling pointer.	Set <code>running_vm = nullptr</code> at the end of <code>cpu()</code> .
original_tio is uninitialised until tcgetattr succeeds	When <code>stdin</code> is not a terminal (a pipe or <code>/dev/null</code>) <code>tcgetattr</code> fails and <code>restore_input_buffering</code> later passes indeterminate bytes to <code>tcsetattr</code> , which also fails harmlessly with <code>ENOTTY</code> .	Value-initialise: <code>struct termios original_tio{}</code> ; and skip restore if the initial <code>tcgetattr</code> failed.
No RAII for the terminal	If any exception is thrown while raw mode is active, <code>restore_input_buffering()</code> is skipped. Today nothing in the loop throws, so the risk is theoretical.	A small guard object whose destructor restores the terminal.

PART V — THE OUTSIDE WORLD

13 Errors, lifecycle, and main()

13.1 main.cpp

src/main.cpp (lines 1-19)

```
1 #include "lc3.hpp"
2 #include <iostream>
3 #include <exception>
4
5 int main(int argc, const char *argv[])
6 {
7     try
8     {
9         lc3 vm;
10        vm.cpu(argc, argv);
11    }
12    catch (const std::exception &e)
13    {
14        std::cerr << "VM Error: " << e.what() << '\n';
15        return 1;
16    }
17
18    return 0;
19 }
```

The entry point is deliberately minimal: it creates the machine, hands it the command line, and turns any exception into a message on standard error and a non-zero exit status. Everything interesting happens inside `vm.cpu(argc, argv)`.

- `catch (const std::exception &e)` catches every standard exception (`invalid_argument`, `runtime_error` and anything derived from `std::exception`). The message goes to `std::cerr`, the stream reserved for diagnostics, so normal program output stays clean when redirected.
- `return 1` follows the Unix convention: 0 means success, non-zero means failure. Scripts can therefore test whether a run succeeded.
- The catch parameter is a *const reference*: it avoids copying the exception object and prevents slicing a derived exception into its base class.

13.2 Exceptions instead of exit()

DESIGN DECISION

Throw, don't terminate. The comment in `cpu()` states the principle: *a vm should never forcefully terminate a host process*. Calling `std::exit()` deep inside a library would kill any program that embedded the VM, skip destructors, and make the failure path untestable. Throwing lets the caller decide what to do, and lets unit tests assert on the exception type (`REQUIRE_THROWS_AS`). As the README puts it, exceptions give 'deterministic fault handling'.

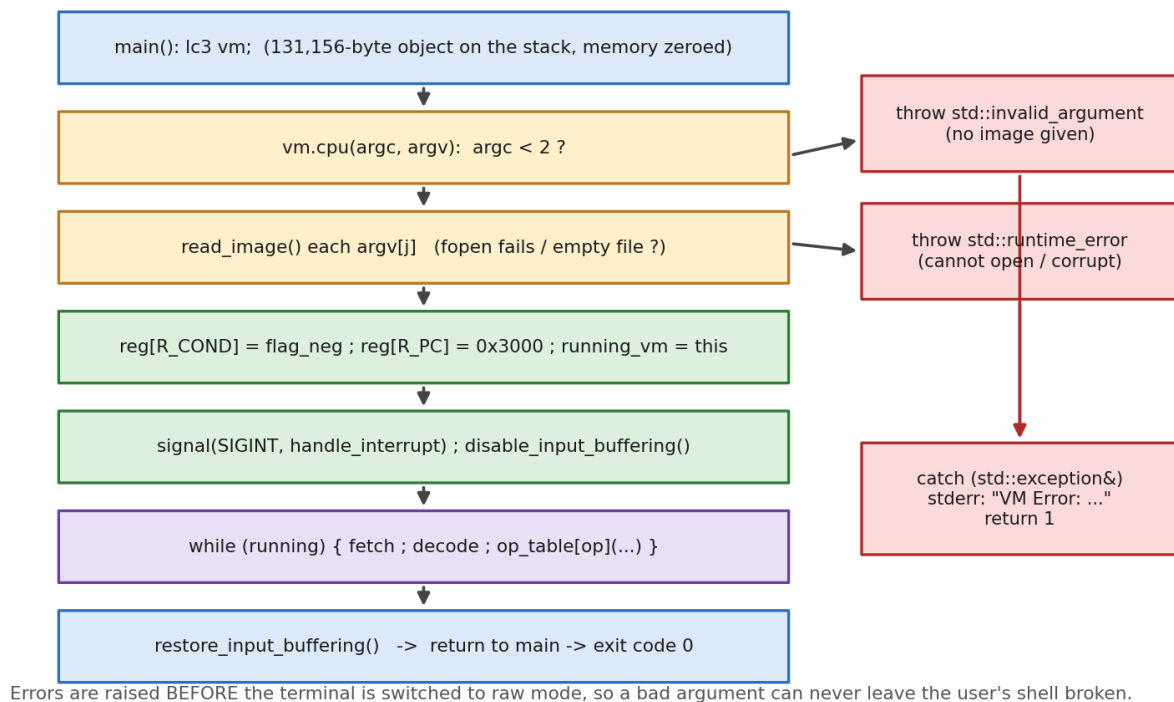


Figure 13.1 — The life of a run, and the two places errors can leave it.

Situation	Where detected	Exception	Message on stderr	Exit code
No image argument	cpu(), first line	std::invalid_argument	VM Error: lc3 [image_file not found]	1
File cannot be opened	read_image returns 0	std::runtime_error	VM Error: Unable to load nope.obj	1
Empty / too-short file	read_image_file	std::runtime_error	VM Error: Corrupted or empty image file	1
HALT executed	TRAP handler	none	(banner on stdout)	0
Ctrl-C	signal handler	none	(banner on stdout)	0
Opcode 8 or 13	run loop	none (machine just stops)	—	0

VERIFIED HERE

The three error rows were produced by running the real binary: with no arguments, with `nope.obj`, and with a 1-byte file. Each printed exactly the message shown and returned exit status 1. Because all three fail *before* the terminal is switched to raw mode, the shell is never disturbed.

NOTE

The message text "`lc3 [image_file not found]`" is a little misleading: it is thrown when *no argument* is given, not when a file is missing. Something like "`usage: lc3 image.obj [...]`" would be clearer to users.

PART VI — VERIFICATION

14 Testing with Catch2

An emulator is a tempting thing to test badly: run a big program, and if the screen looks right, declare victory. This project takes the harder and better route — **small, precise tests that run one instruction and inspect the exact state afterwards**. If the ADD test passes, ADD is right; if a game misbehaves, you already know the primitives are sound.

14.1 Catch2 in five minutes

tests/test_main.cpp (lines 1-6)

```
1 #define CATCH_CONFIG_MAIN
2 #include <catch2/catch.hpp>
3 #include "lc3.hpp"
4 #include <cstdio>
5 #include <sstream>
6 #include <string>
```

Construct	Meaning
#define CATCH_CONFIG_MAIN	Must precede the include in <i>exactly one</i> file: Catch2 then generates main() for you (argument parsing, running tests, printing the summary).
TEST_CASE("name", "[tag]")	Defines a test. It registers itself automatically; no list of tests is maintained. Tags let you run subsets: ./unit_tests "[alu]".
REQUIRE(expr)	Asserts expr is true; on failure Catch2 prints the expression with the values of both sides and aborts that test case.
SECTION("name")	A branch inside a test case. Catch2 runs the test case once per section, from the top each time, executing only that one section. Statements before the sections are therefore the <i>fixture</i> : each section gets a fresh machine.
REQUIRE_THROWS_AS(expr, T)	Asserts that evaluating expr throws an exception of type T.

CONCEPT

Why sections matter here. In the *ALU & Data Processing* test, `lc3 machine`; and the PC assignment sit above three sections. Because the test case restarts for each section, the ADD-register section can never see registers left over from the NOT section. Test isolation comes for free.

14.2 The eleven test cases, decoded

The tests write raw hexadecimal machine words into memory, so the constants deserve decoding. Every row below was decoded by hand using Figure 2.3 and then confirmed by the tests passing.

Test case (tags)	What it proves	Assertions
LC3 Core Hardware Allocation Validation [core]	R_COUNT is 10; the very first (index 0) and very last (index 65535) memory words can be written and read back, so the array really spans the whole address space.	3
ALU Bitwise Math Validation [alu]	sign_extend for 4- and 5-bit fields at the boundaries: 0, +max, -min, -1 (see Chapter 5).	5
Flag updatation [flag]	update_flags yields Z for 0x0000, N for 0x8000, P for 0x0001.	3
Endian Byte Swap Validation [pipeline]	swap(0x1234)=0x3412, swap(0x00FF)=0xFF00, swap(0xABCD)=0xCDAB.	3
Image Reading and File Loading [io]	Writes a real .obj (origin 0x3000, word 0x1234), calls read_image, checks memory[0x3000] and the return value.	3
ALU & Data Processing [alu][opcodes]	ADD register, ADD immediate, NOT — result <i>and</i> flag after one step().	6

Test case (tags)	What it proves	Assertions
Memory & Addressing Modes [memory][opcodes]	LD (PC-relative), LDR (base+offset), LEA (address only).	3
Control Flow & Branching [control][opcodes]	BRz taken, JMP, JSR (link register and target).	4
System Calls & Memory-Mapped I/O [io][trap]	TRAP HALT clears running (stdout captured to keep the test log quiet).	1
CPU Initialization Exceptions [exceptions]	cpu() throws std::invalid_argument with no image and std::runtime_error for a missing file.	2
Clean Loop Termination and Console Output [execution][trap]	End-to-end: builds a one-instruction .obj (TRAP HALT), runs the whole cpu() loop, checks that running is 0 and the banner was printed.	3

Hand-decoding the machine words

Word	Binary (op fields)	Instruction	Why the expected result is right
0x1042	0001 000 001 0 00 010	ADD R0,R1,R2	5 + 3 = 8; positive flag
0x107D	0001 000 001 1 11101	ADD R0,R1,#-3	imm5 = 11101 = -3; 10 - 3 = 7
0x907F	1001 000 001 111111	NOT R0,R1	~0x00FF = 0xFF00 (sign bit set: N)
0x2005	0010 000 000000101	LD R0,+5	PC after fetch 0x3001 + 5 = 0x3006 (test puts 0x9999 there)
0x6045	0110 000 001 000101	LDR R0,R1,#5	R1 = 0x4000, + 5 = 0x4005
0xE005	1110 000 000000101	LEA R0,+5	R0 = 0x3001 + 5 = 0x3006 (an address)
0x0404	0000 0 1 0 000000100	BRz +4	z bit set and COND = Z: PC = 0x3001 + 4 = 0x3005
0xC040	1100 000 001 000000	JMP R1	PC = R1 = 0x5000
0x480A	0100 1 00000001010	JSR +10	R7 = 0x3001; PC = 0x3001 + 10 = 0x300B
0xF025	1111 0000 00100101	TRAP x25	HALT

14.3 Techniques worth copying

- **Direct memory injection.** `machine.memory[0x3000] = 0x1042`; puts an instruction into RAM with no assembler, file or I/O involved. The test is instantaneous and completely deterministic (the README calls this "decoupled from disk I/O latency").
- **One step, then inspect.** Because `step()` exists (Chapter 7) each test isolates a single instruction; a failing assertion points at exactly one handler.
- **Capturing output.** The trap tests swap the stream buffer of `std::cout` for a `std::stringstream` and restore it afterwards: `std::cout.rdbuf(buffer.rdbuf())`. This both silences the log and lets the test *read* what the VM printed.
- **Real files for the loader.** The loader is about files, so its tests write a real temporary .obj with `fwrite` (deliberately swapping the words to big-endian first) and delete it afterwards.
- **Exception assertions.** `REQUIRE_THROWS_AS` checks the *type* of exception, not just that something went wrong.

NOTE

Two small robustness points in the tests themselves: if a `REQUIRE` fails *between* redirecting `std::cout` and restoring it, later output would go to the stringstream; a tiny RAIL guard (see `CoutCapture` in Appendix E) prevents that. Likewise a failing assertion before `remove(test_filename)` leaves the temporary .obj file behind.

14.4 Results

VERIFIED HERE

The original suite, linked against the same library and run for this book (with a stand-in for Catch2's header, see Chapter 3):

```
[ PASS ] LC3 Core Hardware Allocation Validation [core]
[ PASS ] ALU Bitwise Math Validation [alu]
[ PASS ] Flag updation [flag]
[ PASS ] Endian Byte Swap Validation [pipeline]
[ PASS ] Image Reading and File Loading [io]
[ PASS ] ALU & Data Processing [alu][opcodes]
[ PASS ] Memory & Addressing Modes [memory][opcodes]
[ PASS ] Control Flow & Branching [control][opcodes]
[ PASS ] System Calls & Memory-Mapped I/O [io][trap]
[ PASS ] CPU Initialization Exceptions [exceptions]
[ PASS ] Clean Loop Termination and Console Output [execution][trap]
```

All tests passed (36 assertions in 11 test cases)

The same suite also passed when the library and tests were compiled with `-fsanitize=address,undefined`, which would have reported memory errors or undefined behaviour such as signed overflow or out-of-bounds indexing.

14.5 What the suite does not cover

Which opcodes does each test file exercise?

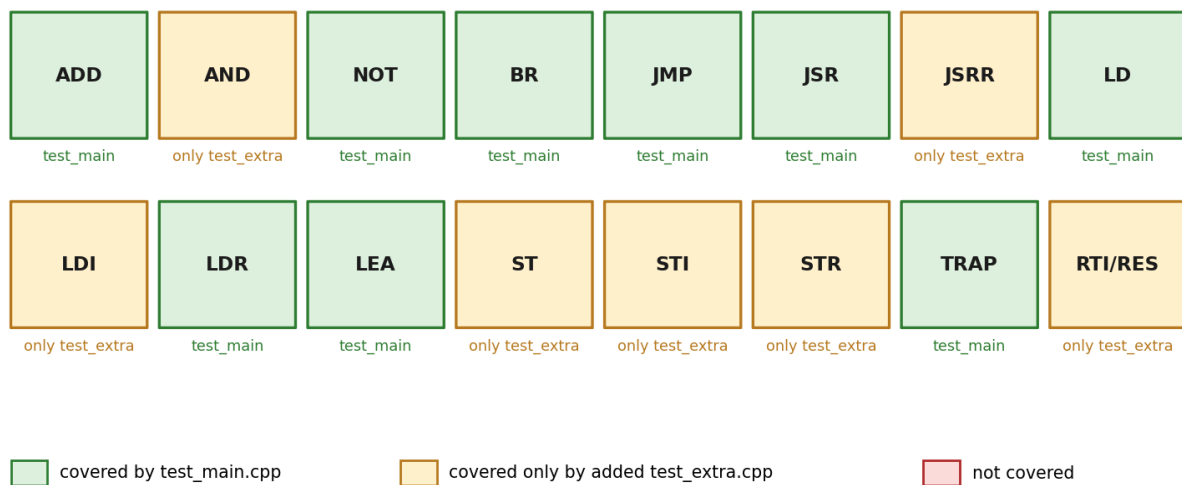


Figure 14.1 — Opcode coverage by test file. Green: original suite. Amber: covered only by the added tests. Red: nothing.

The test case titled *ADD/AND Register Mode* only exercises ADD; AND, LDI, ST, STI, STR and JSRR are never executed by the original suite, and neither are BR-not-taken, PUTS/PUTSP/OUT, the keyboard registers, or the RTI/reserved-opcode stop. These are exactly the places where the review in Chapter 15 found a bug (JSRR). To close the gaps, a second test file was written and run:

Added test case	Covers
AND register and immediate modes	AND reg, AND #-1 (identity), AND #0 (sets Z)
ADD wraps modulo 2^{16} and flags the sign	$0x7FFF + 1 = 0x8000$, flag N
Indirect and store addressing modes	LDI, ST, STI, STR; ST leaves the flags alone
Branch not taken, unconditional, and backward	BRz with flag N falls through; BRnzp #-2 goes to $0x2FFF$
JSRR and RET (JMP R7)	JSRR R2 links and jumps; RET returns to R7

Added test case	Covers
Trap output routines	OUT (low byte only), PUTS, PUTSP, R7 saved by TRAP
Unimplemented opcodes stop the machine cleanly	opcode 8, opcode 13, step() after halt is a no-op
Keyboard status register reads as not-ready without input	KBSR/KBDR read 0 when nothing is pending
Loader honours the origin and big-endian encoding	raw bytes 40 00 AB CD 12 34 land at 0x4000/0x4001

VERIFIED HERE

The added file (Appendix E) compiled against the same library and passed:

```
[ PASS ] AND register and immediate modes [alu][opcodes][extra]
[ PASS ] ADD wraps modulo 2^16 and flags the sign [alu][extra]
[ PASS ] Indirect and store addressing modes [memory][opcodes][extra]
[ PASS ] Branch not taken, unconditional, and backward [control][extra]
[ PASS ] JSRR and RET (JMP R7) [control][extra]
[ PASS ] Trap output routines [io][trap][extra]
[ PASS ] Unimplemented opcodes stop the machine cleanly [control][extra]
[ PASS ] Keyboard status register reads as not-ready without input [io][mmio][extra]
[ PASS ] Loader honours the origin and big-endian encoding [io][extra]
```

All tests passed (29 assertions in 9 test cases)

To use it in the real build, change the test target to `add_executable(unit_tests tests/test_main.cpp tests/test_extra.cpp)`. Only one file may define `CATCH_CONFIG_MAIN`, which is why the added file does not.

WATCH OUT

Two bugs found by review are deliberately *not* in the added tests because the tests would fail today. After fixing them, these are the tests to add:

```
TEST_CASE("JSRR R7 jumps to the old R7 value", "[control]")
{
    lc3 m; m.reg[lc3::R_PC] = 0x3000; m.reg[lc3::R_R7] = 0x5000;
    m.memory[0x3000] = 0x41C0; // JSRR R7
    m.step();
    REQUIRE(m.reg[lc3::R_PC] == 0x5000);
    REQUIRE(m.reg[lc3::R_R7] == 0x3001);
}

TEST_CASE("An image with origin 0 loads", "[io]")
{
    lc3 m; /* write .obj: bytes 00 00 12 34 */
    REQUIRE(m.read_image("o0.obj") == 1);
    REQUIRE(m.memory[0] == 0x1234);
}
```

15 A critical review

Reading code critically is a skill worth practising on code you admire. This chapter lists what the project does well, then every issue found, with how it was established. Nothing here is a criticism of effort: the project is a clean, well-structured first large C++ program, and most items are the kind of subtlety that only appears when someone deliberately looks for it.

15.1 Strengths

- **The right decomposition.** A thin `main()` over a testable library, with all state in one class, makes the VM easy to test and easy to embed.
- **Correct core semantics.** Every one of the 27 worked examples in Chapter 9-10 produced the specified result; all six operand masks match the ISA; PC-relative arithmetic uses the incremented PC; sign extension and wrap-around are right.
- **Careful I/O.** Non-consuming status polls, zero-timeout `select()`, a saved-and-restored terminal, and a SIGINT handler that fixes the terminal — these are exactly the details that make emulator games playable.
- **Fail-safe error handling.** Errors throw before the terminal is touched; `main()` reports and returns 1; unknown opcodes stop instead of crashing.
- **Meaningful tests.** Hand-written machine words, single-stepping, isolated fixtures, exception-type assertions, and captured output.
- **Clear intent in comments.** Remarks such as "a vm should never forcefully terminate a host process" record *why*, which is what future readers need most.

15.2 Findings

'Verified' means reproduced by running code; 'Reasoned' means established by reading the code and language rules.

ID	Finding	How established	Suggested fix
F1	An image with origin 0x0000 loads nothing: <code>max_cap</code> (<code>uint16_t</code>) receives 65,536 and truncates to 0.	Verified: <code>memory[0]</code> stayed 0 after loading word 0x1234 at origin 0.	<code>size_t max_cap = MEMORY_SIZE - orgin;</code>
F2	<code>JSRR R7</code> jumps to PC instead of the old R7 (R7 is overwritten before the base register is read).	Verified: PC = 0x3001, expected 0x5000.	Read the target into a temporary before writing R7.
F3	<code>GETC/IN</code> sign-extend bytes above 0x7F (char is signed).	Verified: <code>(char)0xE9</code> casts to 0xFFE9.	Read into unsigned char.
F4	<code>read()</code> return values ignored; end-of-input yields character 0 forever.	Verified: 29 -Wunused-result warnings. EOF behaviour reasoned.	Check the result; halt or return an EOF marker.
F5	Signal handler calls <code>printf</code> and writes a non-atomic int.	Reasoned (POSIX <code>async-signal-safety</code> rules).	Use <code>write()</code> ; make running volatile <code>std::sig_atomic_t</code> .
F6	<code>running_vm</code> is never reset, so it can dangle after the machine is destroyed.	Reasoned.	Set to <code>nullptr</code> at the end of <code>cpu()</code> .
F7	<code>original_tio</code> is uninitialised when <code>tcgetattr</code> fails (non-tty <code>stdin</code>); it is later passed to <code>tcsetattr</code> .	Reasoned; runs under <code>/dev/null</code> work because <code>tcsetattr</code> just fails with <code>ENOTTY</code> .	<code>struct termios original_tio{};</code> remember whether <code>tcgetattr</code> succeeded.
F8	<code>swap()</code> is applied unconditionally, assuming a little-endian host.	Reasoned.	Assemble words from bytes, or swap only on little-endian hosts.
F9	<code>PUTS/PUTSP</code> bound check <code>addr < MEMORY_SIZE</code> is always true for a <code>uint16_t</code> .	Reasoned (max 65,535 < 65,536).	Remove it, or bound the loop differently.

ID	Finding	How established	Suggested fix
F10	cpu() and step() duplicate the fetch/dispatch code.	Observed in source.	while (running) step();
F11	read_image leaks the FILE* if read_image_file throws.	Reasoned (fclose follows the call).	RAII wrapper.
F12	read_image_file uses u_int16_t (a BSD/glibc typedef) instead of uint16_t; several included headers are unused.	Observed in source; compiles on Linux.	Use uint16_t; trim includes.
F13	R_COND is 0 (no flag) on a fresh object, and cpu() sets N at start.	Verified: printed 0 before cpu(). Tests never rely on it.	Document the choice, or initialise in the class.
F14	The usage exception text says image_file not found when none was given.	Verified (CLI output).	Message like: usage: lc3 image.obj [...]
F15	Tests can leave stdout redirected or a temp file behind on failure.	Reasoned.	RAII guards.

15.3 README versus code

The README is enthusiastic and mostly accurate. A few phrases describe more than the code does; noting them helps readers calibrate:

README says	The code shows
Engineered in C++20	CMakeLists sets CMAKE_CXX_STANDARD 17, and nothing newer than C++17 is used.
Cycle-accurate execution / cycle-by-cycle stepping	Instruction-accurate: step() runs one whole instruction (see Chapter 1).
Cache-aligned memory, rigid bounds checking	No alignas or explicit alignment appears. Bounds safety comes from using uint16_t as the index type; the only explicit check (in PUTS) cannot fail.
Bypasses stream buffering for a more authentic hardware feel	Input uses read() on fd 0; output still goes through std::cout, flushed after each call.
Complete LC-3 ISA	Everything except RTI and the interrupt/privilege machinery; no display registers (DSR/DDR).
Step 1 shows the clone URL as x14-engine.git	The link target (and the repository) is LC3.git; use that.

DESIGN DECISION

Priorities if you have one evening. (1) Fix F2 and F1 and add their tests — they are the only functional deviations from the ISA. (2) Fix F3/F4 for robust input. (3) Make the signal path safe (F5-F7) with three tiny edits. (4) Deduplicate cpu()/step() (F10) so any future feature goes in one place.

PART VI — VERIFICATION

16 Extending the machine: exercises

The best way to learn a codebase is to change it. Each exercise below is small, testable, and touches one mechanism from this book.

#	Exercise	Skills	Hint
1	Fix the origin-0 loader bug (F1) and add its test.	types, pointer arithmetic, Catch2	Chapter 6; use <code>size_t</code> and re-run the probe.
2	Fix JSRR R7 (F2) and add the test from Chapter 14.	ISA semantics	Compute the target before writing R7.
3	Make <code>cpu()</code> call <code>step()</code> so the loop exists once.	refactoring	Watch that <code>step()</code> checks running.
4	Add an instruction counter and a <code>--trace</code> flag that prints PC, instruction and registers each step.	CLI, decoding	Use the field extraction from Chapter 9; the tracer used for Figure 7.2 is 8 lines.
5	Write a disassembler: given a word, print its mnemonic.	bit fields	Switch on <code>instr >> 12</code> ; reuse the formats in Figure 2.3.
6	Convert the mask guards to <code>if constexpr</code> and compare object sizes and warnings.	C++17 templates	Chapter 8.6; run <code>nm -S</code> before and after.
7	Implement DSR/DDR display registers (0xFE04/0xFE06) via a hook in <code>memory_write</code> .	memory-mapped I/O	Chapter 11; DSR bit 15 is always ready.
8	Make end-of-input halt the machine instead of returning 0 forever.	POSIX I/O	<code>read()</code> returns 0 at EOF.
9	Add a <code>RAII TerminalMode</code> class whose destructor restores the terminal.	RAII, exception safety	Chapter 12.3.
10	Run a real assembled program (for example a community 2048.obj) and add an end-to-end smoke test.	integration testing	Feed keys through a pty as in Chapter 11.

APPENDICES

A Index of design decisions

Every deliberate choice found in the code, with where it is discussed.

#	Decision	Where
1	Split into a static library plus thin executables	3.1
2	Pin Catch2 to v2.13.9 through FetchContent	3
3	Public data members to allow white-box tests	4.2
4	Memory as an in-object array, zero-initialised, no heap	4.2
5	R_COUNT sentinel enumerator sizes the register array	4.2
6	Anonymous enums (compile-time constants) for registers, opcodes, flags, MMIO, traps	4.3
7	Flag bit order mirrors the n-z-p bits of BR	4.3
8	sign_extend by copying the sign bit upward with 0xFFFFU	5.1
9	update_flags tests zero first and assigns (not ORs) the flag	5.2
10	swap() to convert big-endian images on a little-endian host	5.3, 6.1
11	fread straight into memory, then swap in place	6.2
12	Execution always starts at 0x3000	7.1
13	PC incremented during fetch (post-increment)	7.2
14	Unknown opcodes stop the machine instead of crashing	7.2, 9.5
15	A separate step() for single-stepping and tests	7.3
16	op_table: array of function pointers, O(1) dispatch	8.1
17	One function template instantiated per opcode	8.2
18	Bit-mask guards choose which decode blocks each opcode keeps	8.3
19	Effective addresses computed once in the prologue	9.1, 9.3
20	Shared flag-update epilogue (mask 0x4666)	9.6
21	TRAP routines implemented as host code, R7 still saved	10, 10.1
22	read() on fd 0 instead of std::cin	10.2
23	std::flush after every output call	10.3
24	KBSR poll does not consume a key; KBDR read does	11.2
25	select() with a zero timeout for non-blocking polling	11.3
26	termios: copy, clear ICANON ECHO, VMIN=1/VTIME=0, restore the original	12.1
27	SIGINT handler restores the terminal and stops the loop	12.2
28	Throw exceptions instead of calling exit(); catch only in main()	13.1, 13.2
29	Errors raised before the terminal enters raw mode	13.2
30	Tests: direct memory injection and one step() per instruction	14.3
31	Tests: capture std::cout with rdbuf redirection	14.3

B Cheat sheets

B.1 Hex, binary and powers of two

Hex	Binary	Dec	Hex	Binary	Dec
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

Useful values: $2^{16} = 65,536$ (address space); 0x8000 = bit 15 (the sign bit and the keyboard ready bit); 0xFFFF = all ones = -1; 0x3000 = 12,288 (program start).

B.2 Field extraction recipes used in the code

Field	Bits	C++ expression	Width mask
opcode	15-12	<code>instr >> 12</code>	—
DR / SR	11-9	<code>(instr >> 9) & 0x7</code>	0x7
SR1 / BaseR	8-6	<code>(instr >> 6) & 0x7</code>	0x7
SR2	2-0	<code>instr & 0x7</code>	0x7
imm5 mode flag	5	<code>(instr >> 5) & 0x1</code>	0x1
imm5	4-0	<code>sign_extend(instr & 0x1F, 5)</code>	0x1F
offset6	5-0	<code>sign_extend(instr & 0x3F, 6)</code>	0x3F
PCoffset9	8-0	<code>sign_extend(instr & 0x1FF, 9)</code>	0x1FF
PCoffset11	10-0	<code>sign_extend(instr & 0x7FF, 11)</code>	0x7FF
JSR mode	11	<code>(instr >> 11) & 1</code>	0x1
BR n z p	11-9	<code>(instr >> 9) & 0x7</code>	0x7
trapvect8	7-0	<code>instr & 0xFF</code>	0xFF

B.3 Hand-assembling a program (the helper used for this book)

No assembler is needed to try the VM. This 20-line Python helper encodes the instructions used in Chapter 7 and writes a big-endian .obj; it produced hello.obj and countdown.obj.

```
import struct
def obj(path, origin, words):
    with open(path, 'wb') as f:
        f.write(struct.pack('>H', origin)) # big-endian origin
        for w in words: f.write(struct.pack('>H', w & 0xFFFF))
def s(v, bits): return v & ((1 << bits) - 1) # two's-complement field
def ADD_i(dr, sr, imm): return (1<<12)|(dr<<9)|(sr<<6)|(1<<5)|s(imm,5)
def ADD_r(dr, a, b): return (1<<12)|(dr<<9)|{a<<6}|b
def AND_i(dr, sr, imm): return (5<<12)|(dr<<9)|(sr<<6)|(1<<5)|s(imm,5)
def BR(n,z,p,off): return (n<<11)|(z<<10)|(p<<9)|s(off,9)
def LEA(dr, off): return (14<<12)|(dr<<9)|s(off,9)
def LD(dr, off): return (2<<12)|(dr<<9)|s(off,9)
def TRAP(v): return (15<<12)|v

msg = "Hello, LC-3!\n"
```

```
obj('hello.obj', 0x3000, [LEA(0,2), TRAP(0x22), TRAP(0x25)] + [ord(c) for c in msg] + [0])
```

VERIFIED HERE

Running `./lc3 hello.obj` printed: 'Hello, LC-3!\n\n-- HALT EXECUTED --'

C Glossary

Term	Meaning
ALU	Arithmetic-logic unit: the CPU part that adds, ANDs and inverts. Here it is just C++ operators inside <code>ins<op></code> .
Async-signal-safe	A function that is safe to call from inside a signal handler (<code>write</code> is, <code>printf</code> is not).
Big-endian	Most significant byte stored first. LC-3 .obj files use it.
Catch2	A C++ unit-testing framework; v2 ships as a single header.
Condition codes	The N, Z, P flags remembering the sign of the last register write.
Emulator / VM	A program that imitates another machine's hardware.
Endianness	The order in which the bytes of a multi-byte number are stored.
Fetch-decode-execute	The CPU's basic loop: read instruction, interpret it, carry it out.
Instantiation	The compiler generating a concrete function from a template for specific parameters.
Integer promotion	Small integer types are widened to <code>int</code> before arithmetic in C++.
KBSR / KBDR	Keyboard status register (0xFE00) and data register (0xFE02).
Link register	R7: holds the return address after JSR or TRAP.
Little-endian	Least significant byte first: x86-64 and most ARM systems.
Memory-mapped I/O	Device registers accessed through ordinary memory addresses.
Opcode	The 4-bit field (bits 15-12) that selects the operation.
PC-relative	An address formed by adding a small signed offset to the program counter.
Polling	Repeatedly checking whether a device is ready.
Raw mode	Terminal mode delivering keys immediately without line editing or echo.
RAII	Resource Acquisition Is Initialization: acquire in a constructor, release in the destructor.
select()	POSIX call that reports which file descriptors are ready for I/O.
Sign extension	Widening a signed number by replicating its sign bit.
Static library	An archive of compiled code linked into executables at build time.
termios	The POSIX terminal-control interface.
Trap vector	The 8-bit code in a TRAP instruction naming an operating-system service.
Two's complement	The standard signed-integer encoding: the top bit has negative weight.
Word	The LC-3's unit of storage: 16 bits.

D Verification log

Everything below was executed while preparing this book. The environment: Ubuntu 24.04, g++ 13.3.0, no CMake and no network access.

D.1 Build

```
g++ -std=c++17 -Wall -Wextra -O2 -Isrc -c src/lc3.cpp -o lc3.o          # library object
ar rcs liblc3_core.a lc3.o                                           # STATIC library
g++ -std=c++17 -Isrc -I<stand-in> tests/test_main.cpp liblc3_core.a -o unit_tests
g++ -std=c++17 -O2 -Isrc src/main.cpp liblc3_core.a -o lc3
```

Compiler diagnostics at `-Wall -Wextra -O2`: 29 warnings of one kind (`-Wunused-result`, the ignored `read()` result, repeated across the 15 instantiations that textually contain the TRAP code) and one `-Wunused-parameter` (the signal parameter). No maybe-uninitialised warnings.

D.2 Tests

```
[ PASS ] LC3 Core Hardware Allocation Validation [core]
[ PASS ] ALU Bitwise Math Validation [alu]
[ PASS ] Flag updation [flag]
[ PASS ] Endian Byte Swap Validation [pipeline]
[ PASS ] Image Reading and File Loading [io]
[ PASS ] ALU & Data Processing [alu][opcodes]
[ PASS ] Memory & Addressing Modes [memory][opcodes]
[ PASS ] Control Flow & Branching [control][opcodes]
[ PASS ] System Calls & Memory-Mapped I/O [io][trap]
[ PASS ] CPU Initialization Exceptions [exceptions]
[ PASS ] Clean Loop Termination and Console Output [execution][trap]
```

All tests passed (36 assertions in 11 test cases)

```
[ PASS ] AND register and immediate modes [alu][opcodes][extra]
[ PASS ] ADD wraps modulo  $2^{16}$  and flags the sign [alu][extra]
[ PASS ] Indirect and store addressing modes [memory][opcodes][extra]
[ PASS ] Branch not taken, unconditional, and backward [control][extra]
[ PASS ] JSRR and RET (JMP R7) [control][extra]
[ PASS ] Trap output routines [io][trap][extra]
[ PASS ] Unimplemented opcodes stop the machine cleanly [control][extra]
[ PASS ] Keyboard status register reads as not-ready without input [io][mmio][extra]
[ PASS ] Loader honours the origin and big-endian encoding [io][extra]
```

All tests passed (29 assertions in 9 test cases)

Sanitizer build (`-fsanitize=address,undefined`): the original suite passed, 36 assertions in 11 test cases, with no reports.

D.3 Probes of suspected edge cases

```
JSRR R7: PC=0x3001 (spec says 0x5000), R7=0x3001
origin 0: memory[0]=0x0000 (expected 0x1234)
initial R_COND before cpu(): 0
sizeof(lc3)=131156 bytes
static_cast<uint16_t>((char)0xE9)=0xFFE9
ADD 0x7FFF+1 -> 0x8000 cond=4 (neg=4)
LDI R0 -> 0xBEEF (expect BEEF)
```

D.4 Command-line behaviour

```
$ ./lc3 hello.obj
Hello, LC-3!

-- HALT EXECUTED --

$ ./lc3 countdown.obj
54321
-- HALT EXECUTED --

$ ./lc3          -> VM Error: lc3 [image_file not found] | exit=1
$ ./lc3 nope.obj -> VM Error: Unable to load nope.obj | exit=1
```

D.5 Interactive behaviour under a pseudo-terminal

Experiment	Result
Poll loop on KBSR then read KBDR; key 'q' typed	output 'q\r\n-- HALT EXECUTED --\r\n', exit 0
TRAP GETC then OUT; key 'z' typed	output 'z\r\n-- HALT EXECUTED --\r\n'
TRAP IN; key 'k' typed	output 'Enter a character: k\r\n-- HALT EXECUTED --\r\n'
Self-branching loop, SIGINT after 0.5 s	stdout '\n-- Execution Interrupted --\n', exit 0

D.6 Limits of this verification

- CMake was not available, so the CMake flow itself was not executed; only its equivalent compiler commands were.
- The real Catch2 could not be downloaded; a small stand-in implementing TEST_CASE, SECTION, REQUIRE and REQUIRE_THROWS_AS was used. Please run the real flow once.
- The suite was run on Linux only. macOS and WSL are expected to work (same POSIX APIs) but were not tested.
- Findings marked 'reasoned' in Chapter 15 come from reading the code and language rules, not from a failing run.

E Added tests: tests/test_extra.cpp

Complete source of the additional test file described in Chapter 14. It compiles and passes against the unmodified library.

```
1 // Additional Catch2 tests covering opcodes/paths that test_main.cpp does not exercise.
2 // Add to CMake with: add_executable(unit_tests tests/test_main.cpp tests/test_extra.cpp)
3 // (only ONE translation unit may define CATCH_CONFIG_MAIN, so this file does not.)
4 #include <catch2/catch.hpp>
5 #include "lc3.hpp"
6 #include <sstream>
7
8 namespace {
9 struct CoutCapture {
10     std::stringstream buf;
11     std::streambuf *old;
12     CoutCapture() : old(std::cout.rdbuf(buf.rdbuf())) {}
13     ~CoutCapture() { std::cout.rdbuf(old); }
14     std::string str() const { return buf.str(); }
15 };
16 }
17
18 TEST_CASE("AND register and immediate modes", "[alu][opcodes][extra]")
19 {
20     lc3 m;
21     m.reg[lc3::R_PC] = 0x3000;
22     SECTION("AND register")
23     {
24         m.reg[lc3::R_R1] = 0x0F0F;
25         m.reg[lc3::R_R2] = 0x00FF;
26         m.memory[0x3000] = 0x5042; // AND R0, R1, R2
27         m.step();
28         REQUIRE(m.reg[lc3::R_R0] == 0x000F);
29         REQUIRE(m.reg[lc3::R_COND] == lc3::flag_pos);
30     }
31     SECTION("AND immediate -1 is identity")
32     {
33         m.reg[lc3::R_R1] = 0x1234;
34         m.memory[0x3000] = 0x507F; // AND R0, R1, #-1
35         m.step();
36         REQUIRE(m.reg[lc3::R_R0] == 0x1234);
37     }
38     SECTION("AND immediate 0 clears and sets Z")
39     {
40         m.reg[lc3::R_R1] = 0xFFFF;
41         m.memory[0x3000] = 0x5060; // AND R0, R1, #0
42         m.step();
43         REQUIRE(m.reg[lc3::R_R0] == 0);
44         REQUIRE(m.reg[lc3::R_COND] == lc3::flag_zro);
45     }
46 }
47
48 TEST_CASE("ADD wraps modulo 2^16 and flags the sign", "[alu][extra]")
49 {
50     lc3 m;
51     m.reg[lc3::R_PC] = 0x3000;
52     m.reg[lc3::R_R1] = 0x7FFF;
53     m.memory[0x3000] = 0x1261; // ADD R1, R1, #1
54     m.step();
55     REQUIRE(m.reg[lc3::R_R1] == 0x8000);
56     REQUIRE(m.reg[lc3::R_COND] == lc3::flag_neg);
57 }
58
59 TEST_CASE("Indirect and store addressing modes", "[memory][opcodes][extra]")
60 {
61     lc3 m;
62     m.reg[lc3::R_PC] = 0x3000;
63     SECTION("LDI follows a pointer")
64     {
65         m.memory[0x3000] = 0xA002; // LDI R0, PC+2
66         m.memory[0x3003] = 0x4000;
67         m.memory[0x4000] = 0xBEEF;
68         m.step();
69         REQUIRE(m.reg[lc3::R_R0] == 0xBEEF);
70     }
71     SECTION("ST writes PC-relative")
72     {
73         m.reg[lc3::R_R0] = 0xAAAA;
74         m.memory[0x3000] = 0x3005; // ST R0, PC+5
75         m.step();
76         REQUIRE(m.memory[0x3006] == 0xAAAA);
77     }
78     SECTION("STI writes through a pointer")
79     {
80         m.reg[lc3::R_R0] = 0x1111;
81         m.memory[0x3000] = 0xB002; // STI R0, PC+2
```

```

82     m.memory[0x3003] = 0x4000;
83     m.step();
84     REQUIRE(m.memory[0x4000] == 0x1111);
85 }
86 SECTION("STR writes base+offset")
87 {
88     m.reg[lc3::R_R0] = 0x2222;
89     m.reg[lc3::R_R1] = 0x4000;
90     m.memory[0x3000] = 0x7045; // STR R0, R1, #5
91     m.step();
92     REQUIRE(m.memory[0x4005] == 0x2222);
93 }
94 SECTION("ST and STR do not change the condition code")
95 {
96     m.reg[lc3::R_COND] = lc3::flag_zro;
97     m.reg[lc3::R_R0] = 0x8000;
98     m.memory[0x3000] = 0x3005; // ST R0, PC+5
99     m.step();
100    REQUIRE(m.reg[lc3::R_COND] == lc3::flag_zro);
101 }
102 }
103
104 TEST_CASE("Branch not taken, unconditional, and backward", "[control][extra]")
105 {
106     lc3 m;
107     m.reg[lc3::R_PC] = 0x3000;
108     SECTION("BRz falls through when the flag is N")
109     {
110         m.reg[lc3::R_COND] = lc3::flag_neg;
111         m.memory[0x3000] = 0x0404; // BRz PC+4
112         m.step();
113         REQUIRE(m.reg[lc3::R_PC] == 0x3001);
114     }
115     SECTION("BRnzp with offset -2 jumps backward")
116     {
117         m.reg[lc3::R_COND] = lc3::flag_pos;
118         m.memory[0x3000] = 0x0FFE; // BRnzp #-2
119         m.step();
120         REQUIRE(m.reg[lc3::R_PC] == 0x2FFF);
121     }
122 }
123
124 TEST_CASE("JSRR and RET (JMP R7)", "[control][extra]")
125 {
126     lc3 m;
127     m.reg[lc3::R_PC] = 0x3000;
128     SECTION("JSRR jumps to the base register and links")
129     {
130         m.reg[lc3::R_R2] = 0x4000;
131         m.memory[0x3000] = 0x4080; // JSRR R2
132         m.step();
133         REQUIRE(m.reg[lc3::R_PC] == 0x4000);
134         REQUIRE(m.reg[lc3::R_R7] == 0x3001);
135     }
136     SECTION("RET returns to the linked address")
137     {
138         m.reg[lc3::R_R7] = 0x1234;
139         m.memory[0x3000] = 0xC1C0; // JMP R7 (RET)
140         m.step();
141         REQUIRE(m.reg[lc3::R_PC] == 0x1234);
142     }
143 }
144
145 TEST_CASE("Trap output routines", "[io][trap][extra]")
146 {
147     lc3 m;
148     m.reg[lc3::R_PC] = 0x3000;
149     SECTION("OUT prints the low byte of R0")
150     {
151         m.reg[lc3::R_R0] = 0x0141; // high byte must be ignored
152         m.memory[0x3000] = 0xF021;
153         CoutCapture cap;
154         m.step();
155         REQUIRE(cap.str() == "A");
156     }
157     SECTION("PUTS prints a one-char-per-word string")
158     {
159         m.reg[lc3::R_R0] = 0x4000;
160         m.memory[0x4000] = 'H';
161         m.memory[0x4001] = 'i';
162         m.memory[0x4002] = '!';
163         m.memory[0x3000] = 0xF022;
164         CoutCapture cap;
165         m.step();
166         REQUIRE(cap.str() == "Hi!");
167     }
168     SECTION("PUTSP unpacks two chars per word")
169     {
170         m.reg[lc3::R_R0] = 0x4000;

```

```

171     m.memory[0x4000] = 0x6948; // 'H' (low) 'i' (high)
172     m.memory[0x4001] = 0x0021; // '!' only
173     m.memory[0x3000] = 0xF024;
174     CoutCapture cap;
175     m.step();
176     REQUIRE(cap.str() == "Hi!");
177 }
178 SECTION("TRAP saves the return address in R7")
179 {
180     m.reg[lc3::R_R0] = 'x';
181     m.memory[0x3000] = 0xF021;
182     CoutCapture cap;
183     m.step();
184     REQUIRE(m.reg[lc3::R_R7] == 0x3001);
185 }
186 }
187
188 TEST_CASE("Unimplemented opcodes stop the machine cleanly", "[control][extra]")
189 {
190     lc3 m;
191     m.reg[lc3::R_PC] = 0x3000;
192     SECTION("RTI (opcode 8)")
193     {
194         m.memory[0x3000] = 0x8000;
195         m.step();
196         REQUIRE(m.running == 0);
197     }
198     SECTION("reserved opcode 13")
199     {
200         m.memory[0x3000] = 0xD000;
201         m.step();
202         REQUIRE(m.running == 0);
203     }
204     SECTION("step() is a no-op once halted")
205     {
206         m.running = 0;
207         m.memory[0x3000] = 0x1261;
208         m.step();
209         REQUIRE(m.reg[lc3::R_PC] == 0x3000);
210     }
211 }
212
213 TEST_CASE("Keyboard status register reads as not-ready without input", "[io][mmio][extra]")
214 {
215     // With stdin at EOF / not a TTY the poll must not block and must not report a key.
216     lc3 m;
217     // We cannot inject a key portably, so only the negative path is asserted here:
218     // reading KBDR when no key is pending yields 0 and leaves KBSR clear.
219     // (Skipped automatically if the test runner's stdin happens to be readable.)
220     if (m.check_key() == 0)
221     {
222         REQUIRE(m.memory_read(lc3::MR_KBSR) == 0);
223         REQUIRE(m.memory_read(lc3::MR_KBDR) == 0);
224     }
225 }
226
227 TEST_CASE("Loader honours the origin and big-endian encoding", "[io][extra]")
228 {
229     lc3 m;
230     const char *name = "extra_image.obj";
231     FILE *f = fopen(name, "wb");
232     REQUIRE(f != nullptr);
233     const unsigned char bytes[] = {0x40, 0x00, 0xAB, 0xCD, 0x12, 0x34}; // origin x4000, words xABCD, x1234
234     fwrite(bytes, 1, sizeof(bytes), f);
235     fclose(f);
236     REQUIRE(m.read_image(name) == 1);
237     REQUIRE(m.memory[0x4000] == 0xABCD);
238     REQUIRE(m.memory[0x4001] == 0x1234);
239     REQUIRE(m.memory[0x4002] == 0x0000);
240     remove(name);
241 }
242

```


F Complete source listings

The five source files, unmodified, with line numbers matching the excerpts in this book.

F.1 CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.15...4.1)
2 project(x14_lc3 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 #Define the virtual target
6 add_library(lc3_core STATIC src/lc3.cpp) #interphase vs dynamic vs static
7
8 #Map the virtual target to the 'src' directory (linking/shows the preprocessor where to look for when it sees
9 » an unknown #include directory eg #include<lc3.hpp>)
10 target_include_directories(lc3_core PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/src)
11
12
13 # Download Catch2 directly from GitHub
14 include(FetchContent)
15 FetchContent_Declare(
16   Catch2
17   GIT_REPOSITORY https://github.com/catchorg/Catch2.git
18   GIT_TAG         v2.13.9
19 )
20 FetchContent_MakeAvailable(Catch2)
21
22 # Assemble the test binary target and link it
23 add_executable(unit_tests tests/test_main.cpp)
24 target_link_libraries(unit_tests PRIVATE lc3_core Catch2)
25
```

The copy currently on GitHub additionally ends with:

```
# Add the main executable target (point src/main.cpp to your actual entry point)
add_executable(lc3 src/main.cpp)

# Link the core VM logic
target_link_libraries(lc3 PRIVATE lc3_core)
```

F.2 src/lc3.hpp

```
1 // zero ai used
2
3 // blueprint
4
5 #pragma once
6
7 #include <cstdio>
8 #include <stdint>
9 #include <csignal>
10 #include <cstdlib>
11 #include <unistd.h>
12 #include <fcntl.h>
13 #include <termios.h>
14 #include <iostream>
15 #include <sys/time.h>
16 #include <sys/types.h>
17 #include <sys/mman.h>
18 #include <sys/select.h>
19
20 constexpr int MEMORY_SIZE = 1 << 16;
21
22 class lc3
23 {
24     // have to impose boundary protection and intilize the registers
25 public:
26     // STORAGE
27
28     uint16_t memory[MEMORY_SIZE] = {0}; // memory array
29
30     // registers
31     enum
32     {
33
34         R_R0 = 0,
35         R_R1,
36         R_R2,
37         R_R3,
38         R_R4,
```

```

39     R_R5,
40     R_R6,
41     R_R7,
42     R_PC, // program counter
43     R_COND, // condition flags
44     R_COUNT // automatically counts the number of registers
45 };
46
47 uint16_t reg[R_COUNT] = {0}; // register array
48
49 // CPU
50
51 // sign extend
52 uint16_t sign_extend(uint16_t x, int bit_count);
53
54 // falg update
55 void update_flags(uint16_t r);
56
57 // cpu (execution loop)
58 void cpu(int argc, const char *argv[]);
59
60 // swap to account for Endianness
61 uint16_t swap(uint16_t x);
62
63 // read the image file (.obj)
64 int read_image(char const *image_path);
65
66 // reading image file into memory
67 void read_image_file(FILE *file);
68
69 void memory_write(uint16_t address, uint16_t val);
70
71 static void handle_interrupt(int signal);
72
73 struct termios original_tio;
74 uint16_t memory_read(uint16_t address);
75 void disable_input_buffering();
76 void restore_input_buffering();
77 uint16_t check_key();
78 void step(); // webassm
79
80 // Opcodes
81
82 enum
83 {
84     OP_BR = 0, // branch
85     OP_ADD, // add
86     OP_LD, // load
87     OP_ST, // store
88     OP_JSR, // jump register
89     OP_AND, // bitwise and
90     OP_LDR, // load register
91     OP_STR, // store register
92     OP_RTI, // unused
93     OP_NOT, // bitwise not
94     OP_LDI, // load indirect
95     OP_STI, // store indirect
96     OP_JMP, // jump
97     OP_RES, // reserved (unused)
98     OP_LEA, // load effective address
99     OP_TRAP // execute trap
100 };
101
102 // conditional flags
103 // enum is a compile-time literal so no overhead
104 enum
105 {
106     flag_pos = 1 << 0,
107     flag_zro = 1 << 1,
108     flag_neg = 1 << 2
109 };
110
111 enum
112 {
113     MR_KBSR = 0xFE00,
114     MR_KBDR = 0xFE02
115 };
116
117 enum
118 {
119     TRAP_GETC = 0x20,
120     TRAP_OUT = 0x21,
121     TRAP_PUTS = 0x22,
122     TRAP_IN = 0x23,
123     TRAP_PUTSP = 0x24,
124     TRAP_HALT = 0x25
125 };
126
127

```

```

128     int running = 1;
129 };
130

```

F.3 src/lc3.cpp

```

1  // active sillcion
2  #include "lc3.hpp"
3
4  lc3 *running_vm = nullptr;
5  // sign extend (2's complement)
6  uint16_t lc3::sign_extend(uint16_t x, int bit_size)
7  {
8
9      if (x >> (bit_size - 1) & 1)
10     {
11
12         x = (x | (0xFFFFU << bit_size));
13     }
14
15     else
16     {
17         x = 0xFFFFU & x;
18     }
19     return x;
20 }
21
22 void lc3::update_flags(uint16_t r)
23 {
24     if (reg[r] == 0)
25     {
26
27         reg[R_COND] = flag_zro;
28     }
29
30     else if (reg[r] >> 15 & 1)
31     {
32
33         reg[R_COND] = flag_neg;
34     }
35
36     else
37     {
38
39         reg[R_COND] = flag_pos;
40     }
41 }
42
43 template <unsigned op>
44 static void ins(lc3 *vm, uint16_t instr)
45 {
46     uint16_t r0, r1, r2, imm5, imm5_flag;
47     uint16_t pc_plus_off, base_plus_off;
48
49     constexpr uint16_t opbit = 1 << op;
50
51     if (0x4EEE & opbit)
52     {
53         r0 = (instr >> 9) & 0x7;
54     }
55     if (0x12F3 & opbit)
56     {
57         r1 = (instr >> 6) & 0x7;
58     }
59     if (0x0022 & opbit)
60     {
61         imm5_flag = (instr >> 5) & 0x1;
62
63         if (imm5_flag)
64         {
65             imm5 = vm->sign_extend(instr & 0x1F, 5);
66         }
67         else
68         {
69             r2 = instr & 0x7;
70         }
71     }
72     if (0x00C0 & opbit)
73     {
74         base_plus_off = vm->reg[r1] + vm->sign_extend(instr & 0x3F, 6);
75     }
76     if (0x4C0D & opbit)
77     {
78         pc_plus_off = vm->reg[lc3::R_PC] + vm->sign_extend(instr & 0x1FF, 9);
79     }
80
81     if (0x0001 & opbit) // BR

```

```

82     {
83         uint16_t cond = (instr >> 9) & 0x7;
84         if (cond & vm->reg[lc3::R_COND])
85         {
86             vm->reg[lc3::R_PC] = pc_plus_off;
87         }
88     }
89
90     if (0x0002 & opbit) // ADD
91     {
92         if (imm5_flag)
93         {
94             vm->reg[r0] = vm->reg[r1] + imm5;
95         }
96         else
97         {
98             vm->reg[r0] = vm->reg[r1] + vm->reg[r2];
99         }
100     }
101
102     if (0x0020 & opbit) // AND
103     {
104         if (imm5_flag)
105         {
106             vm->reg[r0] = vm->reg[r1] & imm5;
107         }
108         else
109         {
110             vm->reg[r0] = vm->reg[r1] & vm->reg[r2];
111         }
112     }
113
114     if (0x0200 & opbit) // Not
115     {
116         vm->reg[r0] = ~vm->reg[r1];
117     }
118
119     if (0x1000 & opbit) // JMP
120     {
121         vm->reg[lc3::R_PC] = vm->reg[r1];
122     }
123
124     if (0x0010 & opbit) // JSR
125     {
126         uint16_t long_flag = ((instr >> 11) & 1);
127         vm->reg[lc3::R_R7] = vm->reg[lc3::R_PC];
128
129         if (long_flag)
130         {
131             pc_plus_off = vm->reg[lc3::R_PC] + vm->sign_extend(instr & 0x7FF, 11);
132             vm->reg[lc3::R_PC] = pc_plus_off;
133         }
134         else
135         {
136             vm->reg[lc3::R_PC] = vm->reg[r1];
137         }
138     }
139
140     if (0x0004 & opbit)
141     {
142         vm->reg[r0] = vm->memory_read(pc_plus_off);
143     } // Ld
144     if (0x0400 & opbit)
145     {
146         vm->reg[r0] = vm->memory_read(vm->memory_read(pc_plus_off));
147     } // ldi
148     if (0x0040 & opbit)
149     {
150         vm->reg[r0] = vm->memory_read(base_plus_off);
151     } // ldr
152     if (0x4000 & opbit)
153     {
154         vm->reg[r0] = pc_plus_off;
155     } // lea
156     if (0x0008 & opbit)
157     {
158         vm->memory_write(pc_plus_off, vm->reg[r0]);
159     } // st
160     if (0x0800 & opbit)
161     {
162         vm->memory_write(vm->memory_read(pc_plus_off), vm->reg[r0]);
163     } // sti
164     if (0x0080 & opbit)
165     {
166         vm->memory_write(base_plus_off, vm->reg[r0]);
167     } // str
168     if (0x8000 & opbit) // trap
169     {
170

```

```

171     vm->reg[lc3::R_R7] = vm->reg[lc3::R_PC];
172
173     switch (instr & 0xFF)
174     {
175     case lc3::TRAP_GETC:
176     {
177         char c = 0;
178         read(STDIN_FILENO, &c, 1); // read directly from os kernel's buffer replacment for cin(idk error)
179         vm->reg[lc3::R_R0] = static_cast<uint16_t>(c);
180         vm->update_flags(lc3::R_R0);
181         break;
182     }
183     case lc3::TRAP_OUT:
184     {
185         std::cout << static_cast<char>(vm->reg[lc3::R_R0] & 0xFF) << std::flush;
186         break;
187     }
188     case lc3::TRAP_PUTS:
189     {
190         uint16_t addr = vm->reg[lc3::R_R0];
191         while (addr < MEMORY_SIZE && vm->memory[addr] != 0)
192         {
193             std::cout << static_cast<char>(vm->memory[addr]);
194             addr++;
195         }
196         std::cout << std::flush;
197         break;
198     }
199
200     case lc3::TRAP_IN:
201     {
202         std::cout << "Enter a character: " << std::flush;
203         char c = 0;
204         read(STDIN_FILENO, &c, 1);
205         std::cout << c << std::flush;
206         vm->reg[lc3::R_R0] = static_cast<uint16_t>(c);
207         vm->update_flags(lc3::R_R0);
208         break;
209     }
210     case lc3::TRAP_PUTSP:
211     {
212         uint16_t addr = vm->reg[lc3::R_R0];
213         while (addr < MEMORY_SIZE && vm->memory[addr] != 0)
214         {
215             char c1 = (vm->memory[addr]) & 0xFF;
216             std::cout << c1;
217             char c2 = (vm->memory[addr]) >> 8;
218             if (c2)
219                 std::cout << c2;
220             addr++;
221         }
222         std::cout << std::flush;
223         break;
224     }
225     case lc3::TRAP_HALT:
226     {
227         std::cout << "\n-- HALT EXECUTED --\n"
228                 << std::flush;
229         vm->running = 0;
230         break;
231     }
232     default:
233         break;
234     }
235 }
236
237 if (0x4666 & opbit)
238 {
239     {
240         vm->update_flags(r0);
241     }
242 }
243
244 static void (*op_table[16])(lc3 *, uint16_t) =
245 {
246     ins<0>, ins<1>, ins<2>, ins<3>,
247     ins<4>, ins<5>, ins<6>, ins<7>,
248     nullptr, ins<9>, ins<10>, ins<11>,
249     ins<12>, nullptr, ins<14>, ins<15>}; // array that holds pointers to fncs
250
251 void lc3::cpu(int argc, const char *argv[])
252 {
253     {
254         if (argc < 2)
255         {
256             throw std::invalid_argument("lc3 [image_file not found]"); // a vm should never forcefully terminate a
257             » host process
258         }
259     }

```

```

259     for (int j = 1; j < argc; j++)
260     {
261         if (!read_image(argv[j]))
262         {
263             throw std::runtime_error(std::string("Unable to load ") + argv[j]);
264         }
265     }
266 }
267
268 reg[R_COND] = flag_neg;
269
270 enum
271 {
272     PC_START = 0x3000
273 };
274 reg[R_PC] = PC_START;
275 running_vm = this;
276 std::signal(SIGINT, lc3::handle_interrupt);
277 disable_input_buffering();
278
279 while (running)
280 {
281     // fetch
282     uint16_t instr = memory_read(reg[R_PC]++);
283     uint16_t op = instr >> 12;
284
285     if (op_table[op] != nullptr)
286     {
287         op_table[op](this, instr);
288     }
289     else
290     {
291         running = 0;
292     }
293 }
294 restore_input_buffering();
295 }
296
297 uint16_t lc3::swap(uint16_t x)
298 {
299
300     return (x << 8) | (x >> 8);
301 }
302
303 int lc3::read_image(char const *image_path)
304 {
305     FILE *file = fopen(image_path, "rb");
306     if (!file)
307         return 0;
308     else
309         read_image_file(file);
310     fclose(file);
311     return 1;
312 }
313
314 void lc3::read_image_file(FILE *file)
315 {
316     uint16_t orgin;
317     if (fread(&orgin, sizeof(orgin), 1, file) != 1) // reads count of elements found
318     {
319         throw std::runtime_error("Corrupted or empty image file ");
320     }
321     orgin = swap(orgin);
322
323     uint16_t max_cap = MEMORY_SIZE - orgin;
324     uint16_t *p = memory + orgin; // advances the pointer by origin elements – not origin bytes (also note
    » pointer decay)
325
326     size_t read = fread(p, sizeof(u_int16_t), max_cap, file);
327
328     while (read-- > 0)
329     {
330         *p = swap(*p);
331         ++p;
332     }
333 }
334
335 void lc3::memory_write(uint16_t address, uint16_t val)
336 {
337     memory[address] = val;
338 }
339
340 uint16_t lc3::memory_read(uint16_t address)
341 {
342     if (address == MR_KBSR)
343     {
344         if (check_key())
345         {
346             memory[MR_KBSR] = 1 << 15; // Set ready bit, but DO NOT consume key yet

```

```

347     }
348     else
349     {
350         memory[MR_KBSR] = 0;
351     }
352 }
353 else if (address == MR_KBDR)
354 {
355     if (check_key())
356     {
357         memory[MR_KBSR] = 0;
358         char c = 0;
359         read(STDIN_FILENO, &c, 1); // Reads directly from the POSIX file descriptor
360         memory[MR_KBDR] = static_cast<uint16_t>(c);
361     }
362     else
363     {
364         memory[MR_KBDR] = 0;
365     }
366 }
367 return memory[address];
368 }
369
370 void lc3::disable_input_buffering()
371 {
372     tcgetattr(STDIN_FILENO, &original_tio);
373     struct termios new_tio = original_tio;
374     new_tio.c_lflag &= ~(ICANON | ECHO);
375     new_tio.c_cc[VMIN] = 1; // Guarantee blocking until 1 char is typed
376     new_tio.c_cc[VTIME] = 0; // No timeout
377     tcsetattr(STDIN_FILENO, TCSANOW, &new_tio);
378 }
379
380 void lc3::restore_input_buffering()
381 {
382     tcsetattr(STDIN_FILENO, TCSANOW, &original_tio);
383 }
384
385 uint16_t lc3::check_key()
386 {
387     fd_set readfds;
388     FD_ZERO(&readfds);
389     FD_SET(STDIN_FILENO, &readfds);
390
391     struct timeval timeout;
392     timeout.tv_sec = 0;
393     timeout.tv_usec = 0;
394     return select(STDIN_FILENO + 1, &readfds, nullptr, nullptr, &timeout) > 0;
395 }
396
397 void lc3::handle_interrupt(int signal)
398 {
399     if (running_vm)
400     {
401         running_vm->restore_input_buffering();
402         running_vm->running = 0;
403     }
404     std::printf("\n-- Execution Interrupted --\n");
405 }
406
407 void lc3::step()
408 {
409     if (!running)
410         return;
411
412     uint16_t instr = memory_read(reg[R_PC]++);
413     uint16_t op = instr >> 12;
414
415     if (op_table[op] != nullptr)
416     {
417         op_table[op](this, instr);
418     }
419     else
420     {
421         running = 0;
422     }
423 }

```

F.4 src/main.cpp

```
1 #include "lc3.hpp"
2 #include <iostream>
3 #include <exception>
4
5 int main(int argc, const char *argv[])
6 {
7     try
8     {
9         lc3 vm;
10        vm.cpu(argc, argv);
11    }
12    catch (const std::exception &e)
13    {
14        std::cerr << "VM Error: " << e.what() << '\n';
15        return 1;
16    }
17
18    return 0;
19 }
```

F.5 tests/test_main.cpp

```
1 #define CATCH_CONFIG_MAIN
2 #include <catch2/catch.hpp>
3 #include "lc3.hpp"
4 #include <cstdio>
5 #include <sstream>
6 #include <string>
7
8 TEST_CASE("LC3 Core Hardware Allocation Validation", "[core]")
9 {
10     lc3 machine;
11
12     REQUIRE(lc3::R_COUNT == 10);
13
14     machine.memory[0] = 0x1234;
15     machine.memory[65535] = 0xABCD;
16
17     REQUIRE(machine.memory[0] == 0x1234);
18     REQUIRE(machine.memory[65535] == 0xABCD);
19 }
20
21 TEST_CASE("ALU Bitwise Math Validation", "[alu]")
22 {
23     lc3 machine;
24
25     uint16_t x = 0x0A;
26     REQUIRE(machine.sign_extend(x, 4) == 0xFFFFA);
27     REQUIRE(machine.sign_extend(0x00, 5) == 0x0000);
28     REQUIRE(machine.sign_extend(0x0F, 5) == 0x000F);
29     REQUIRE(machine.sign_extend(0x10, 5) == 0xFFFF0);
30     REQUIRE(machine.sign_extend(0x1F, 5) == 0xFFFFF);
31 }
32
33 TEST_CASE("Flag updation", "[flag]")
34 {
35     lc3 machine;
36
37     machine.reg[lc3::R_R0] = 0x0000;
38     machine.update_flags(lc3::R_R0);
39     REQUIRE(machine.reg[lc3::R_COND] == lc3::flag_zro);
40
41     machine.reg[lc3::R_R0] = 0x8000;
42     machine.update_flags(lc3::R_R0);
43     REQUIRE(machine.reg[lc3::R_COND] == lc3::flag_neg);
44
45     machine.reg[lc3::R_R0] = 0x00001;
46     machine.update_flags(lc3::R_R0);
47     REQUIRE(machine.reg[lc3::R_COND] == lc3::flag_pos);
48 }
49
50 TEST_CASE("Endian Byte Swap Validation", "[pipeline]")
51 {
52     lc3 machine;
53
54     REQUIRE(machine.swap(0x1234) == 0x3412);
55     REQUIRE(machine.swap(0x00FF) == 0xFF00);
56     REQUIRE(machine.swap(0xABCD) == 0xCDAB);
57 }
58
59 TEST_CASE("Image Reading and File Loading", "[io]")
60 {
61     lc3 machine;
62 }
```



```

63     const char *test_filename = "test_image.obj";
64     FILE *f = fopen(test_filename, "wb");
65     REQUIRE(f != nullptr);
66
67     uint16_t origin = machine.swap(0x3000);
68     fwrite(&origin, sizeof(origin), 1, f);
69
70     uint16_t sample_data = machine.swap(0x1234);
71     fwrite(&sample_data, sizeof(sample_data), 1, f);
72     fclose(f);
73
74     REQUIRE(machine.read_image(test_filename) == 1);
75     REQUIRE(machine.memory[0x3000] == 0x1234);
76
77     remove(test_filename);
78 }
79
80 TEST_CASE("ALU & Data Processing", "[alu][opcodes]")
81 {
82     lc3 machine;
83     machine.reg[lc3::R_PC] = 0x3000;
84
85     SECTION("ADD/AND Register Mode")
86     {
87         machine.reg[lc3::R_R1] = 5;
88         machine.reg[lc3::R_R2] = 3;
89         machine.memory[0x3000] = 0x1042; // ADD R0, R1, R2
90
91         machine.step();
92
93         REQUIRE(machine.reg[lc3::R_R0] == 8);
94         REQUIRE(machine.reg[lc3::R_COND] == lc3::flag_pos);
95     }
96
97     SECTION("ADD/AND Immediate Mode")
98     {
99         machine.reg[lc3::R_R1] = 10;
100        machine.memory[0x3000] = 0x107D; // ADD R0, R1, #-3
101
102        machine.step();
103
104        REQUIRE(machine.reg[lc3::R_R0] == 7);
105        REQUIRE(machine.reg[lc3::R_COND] == lc3::flag_pos);
106    }
107
108    SECTION("NOT Inversion")
109    {
110        machine.reg[lc3::R_R1] = 0x00FF;
111        machine.memory[0x3000] = 0x907F; // NOT R0, R1
112
113        machine.step();
114
115        REQUIRE(machine.reg[lc3::R_R0] == 0xFF00);
116        REQUIRE(machine.reg[lc3::R_COND] == lc3::flag_neg);
117    }
118 }
119
120 TEST_CASE("Memory & Addressing Modes", "[memory][opcodes]")
121 {
122     lc3 machine;
123     machine.reg[lc3::R_PC] = 0x3000;
124
125     SECTION("LD Direct Addressing")
126     {
127         machine.memory[0x3000] = 0x2005; // LD R0, PC+5
128         machine.memory[0x3006] = 0x9999;
129
130         machine.step();
131
132         REQUIRE(machine.reg[lc3::R_R0] == 0x9999);
133     }
134
135     SECTION("LDR Base+Offset Addressing")
136     {
137         machine.reg[lc3::R_R1] = 0x4000;
138         machine.memory[0x3000] = 0x6045; // LDR R0, R1, #5
139         machine.memory[0x4005] = 0x8888;
140
141         machine.step();
142
143         REQUIRE(machine.reg[lc3::R_R0] == 0x8888);
144     }
145
146     SECTION("LEA Effective Address Loading")
147     {
148         machine.memory[0x3000] = 0xE005; // LEA R0, PC+5
149
150         machine.step();
151     }

```

```

152     REQUIRE(machine.reg[lc3::R_R0] == 0x3006);
153 }
154 }
155
156 TEST_CASE("Control Flow & Branching", "[control][opcodes]")
157 {
158     lc3 machine;
159     machine.reg[lc3::R_PC] = 0x3000;
160
161     SECTION("BR Condition Code permutations")
162     {
163         machine.reg[lc3::R_COND] = lc3::flag_zro;
164         machine.memory[0x3000] = 0x0404; // BRz PC+4
165
166         machine.step();
167
168         REQUIRE(machine.reg[lc3::R_PC] == 0x3005);
169     }
170
171     SECTION("JMP Absolute Jumps")
172     {
173         machine.reg[lc3::R_R1] = 0x5000;
174         machine.memory[0x3000] = 0xC040; // JMP R1
175
176         machine.step();
177
178         REQUIRE(machine.reg[lc3::R_PC] == 0x5000);
179     }
180
181     SECTION("JSR Link Register (R7) Saving")
182     {
183         machine.memory[0x3000] = 0x480A; // JSR PC+10
184
185         machine.step();
186
187         REQUIRE(machine.reg[lc3::R_R7] == 0x3001);
188         REQUIRE(machine.reg[lc3::R_PC] == 0x300B);
189     }
190 }
191
192 TEST_CASE("System Calls & Memory-Mapped I/O", "[io][trap]")
193 {
194     lc3 machine;
195     machine.reg[lc3::R_PC] = 0x3000;
196
197     SECTION("TRAP_HALT Execution Toggle")
198     {
199         machine.memory[0x3000] = 0xF025; // TRAP_HALT
200
201         // Redirect stdout to prevent Catch2 console clutter during testing
202         std::stringstream buffer;
203         std::streambuf *old_cout = std::cout.rdbuf(buffer.rdbuf());
204
205         machine.step();
206
207         std::cout.rdbuf(old_cout);
208
209         REQUIRE(machine.running == 0);
210     }
211 }
212
213 TEST_CASE("CPU Initialization Exceptions", "[exceptions]")
214 {
215     lc3 machine;
216
217     SECTION("Missing image argument throws invalid_argument")
218     {
219         const char *args[] = {"lc3-vm"};
220         REQUIRE_THROWS_AS(machine.cpu(1, args), std::invalid_argument);
221     }
222
223     SECTION("Invalid file path throws runtime_error")
224     {
225         const char *args[] = {"lc3-vm", "fake_corrupted_file.obj"};
226         REQUIRE_THROWS_AS(machine.cpu(2, args), std::runtime_error);
227     }
228 }
229
230 TEST_CASE("Clean Loop Termination and Console Output", "[execution][trap]")
231 {
232     lc3 machine;
233
234     const char *test_filename = "halt_test.obj";
235     FILE *f = fopen(test_filename, "wb");
236     REQUIRE(f != nullptr);
237
238     uint16_t origin = machine.swap(0x3000);
239     fwrite(&origin, sizeof(origin), 1, f);
240

```

```

241     uint16_t halt_instr = machine.swap(0xF025); // TRAP_HALT instruction
242     fwrite(&halt_instr, sizeof(halt_instr), 1, f);
243     fclose(f);
244
245     std::stringstream buffer;
246     std::streambuf *old_cout = std::cout.rdbuf(buffer.rdbuf());
247
248     const char *args[] = {"lc3-vm", test_filename};
249     machine.cpu(2, args);
250
251     std::cout.rdbuf(old_cout);
252
253     REQUIRE(machine.running == 0);
254     REQUIRE(buffer.str().find("-- HALT EXECUTED --") != std::string::npos);
255
256     remove(test_filename);
257 }

```